UVA CS 4774: Machine Learning

S3: Lecture 19: Training Transformers

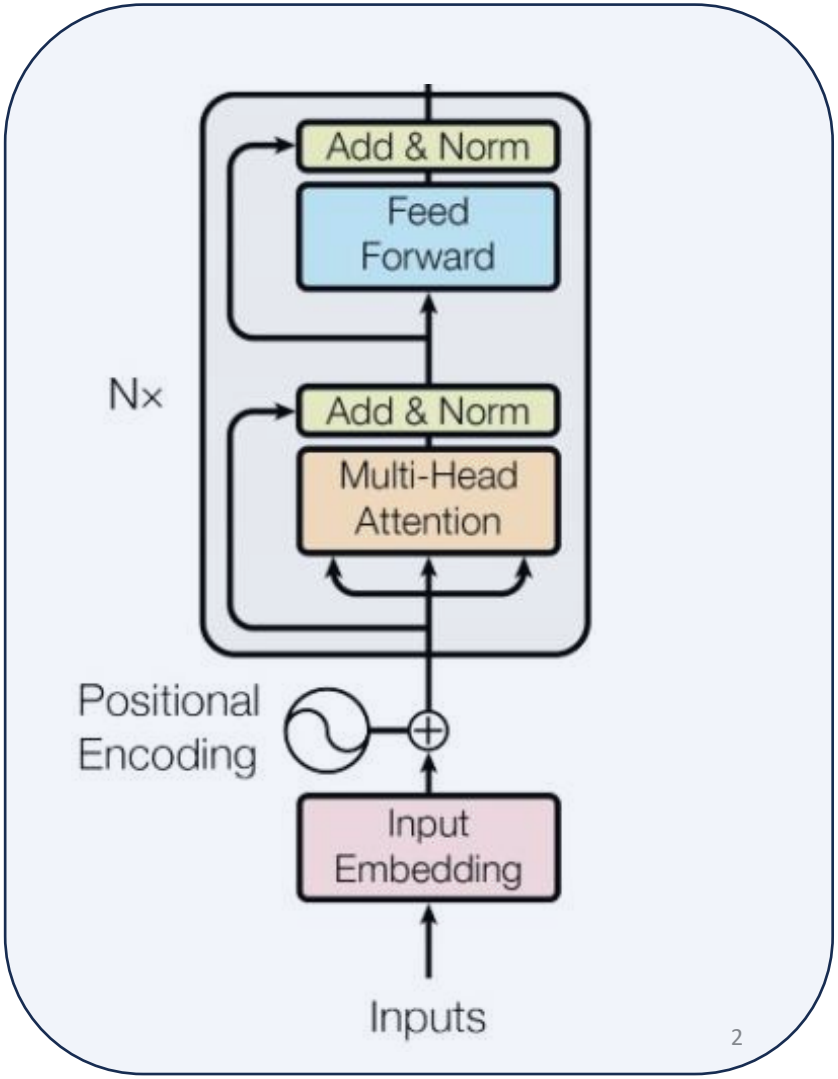
Pretraining & Post-training

Dr. Yanjun Qi

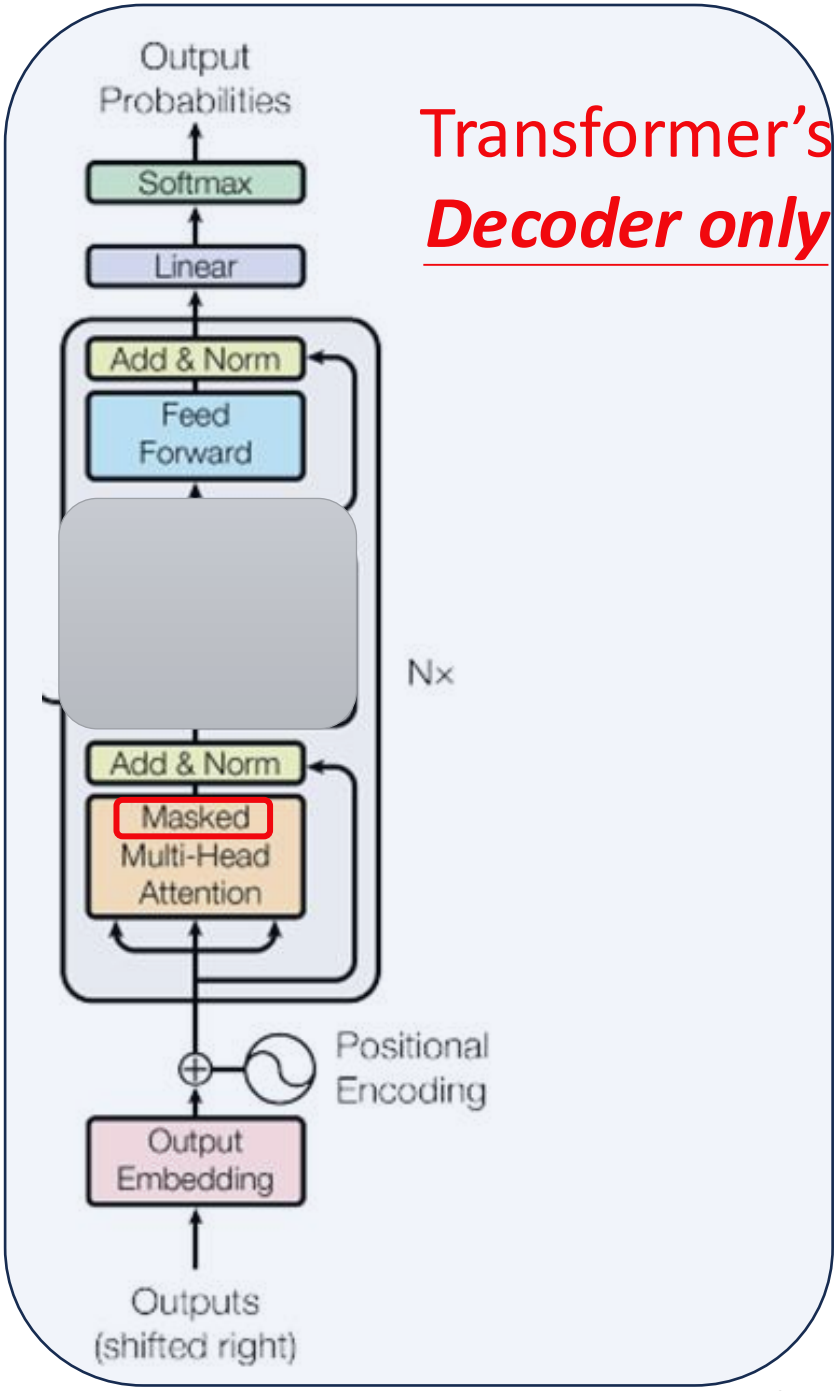
University of Virginia
Department of Computer Science

Recap:

Transformer's ***Encoder Only***



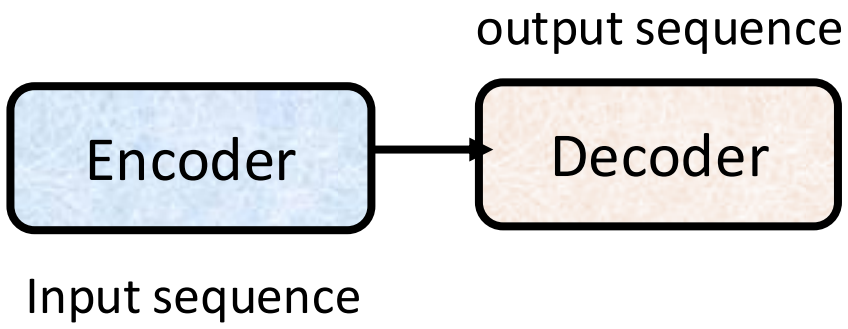
Transformer's ***Decoder only***

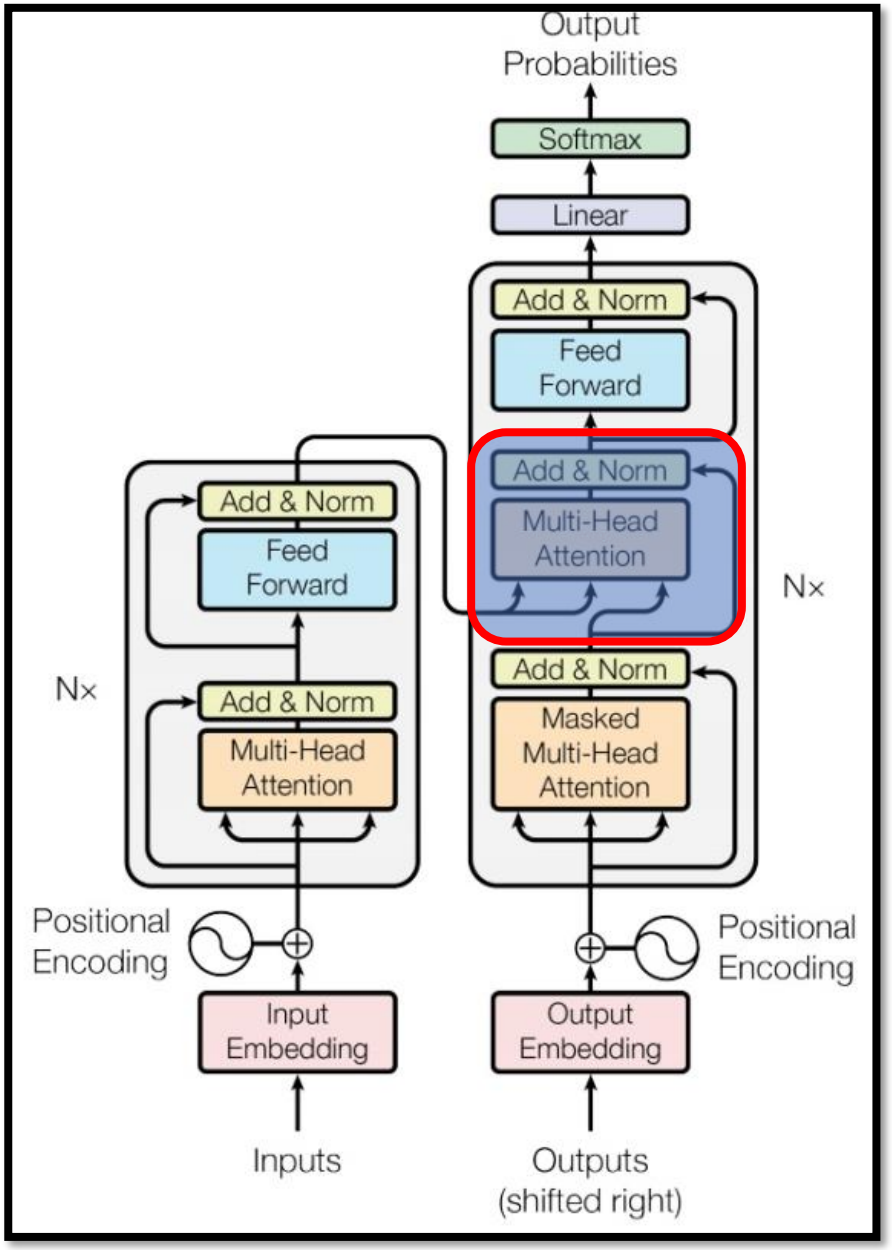


credit: Prof. Hung-yi Lee,

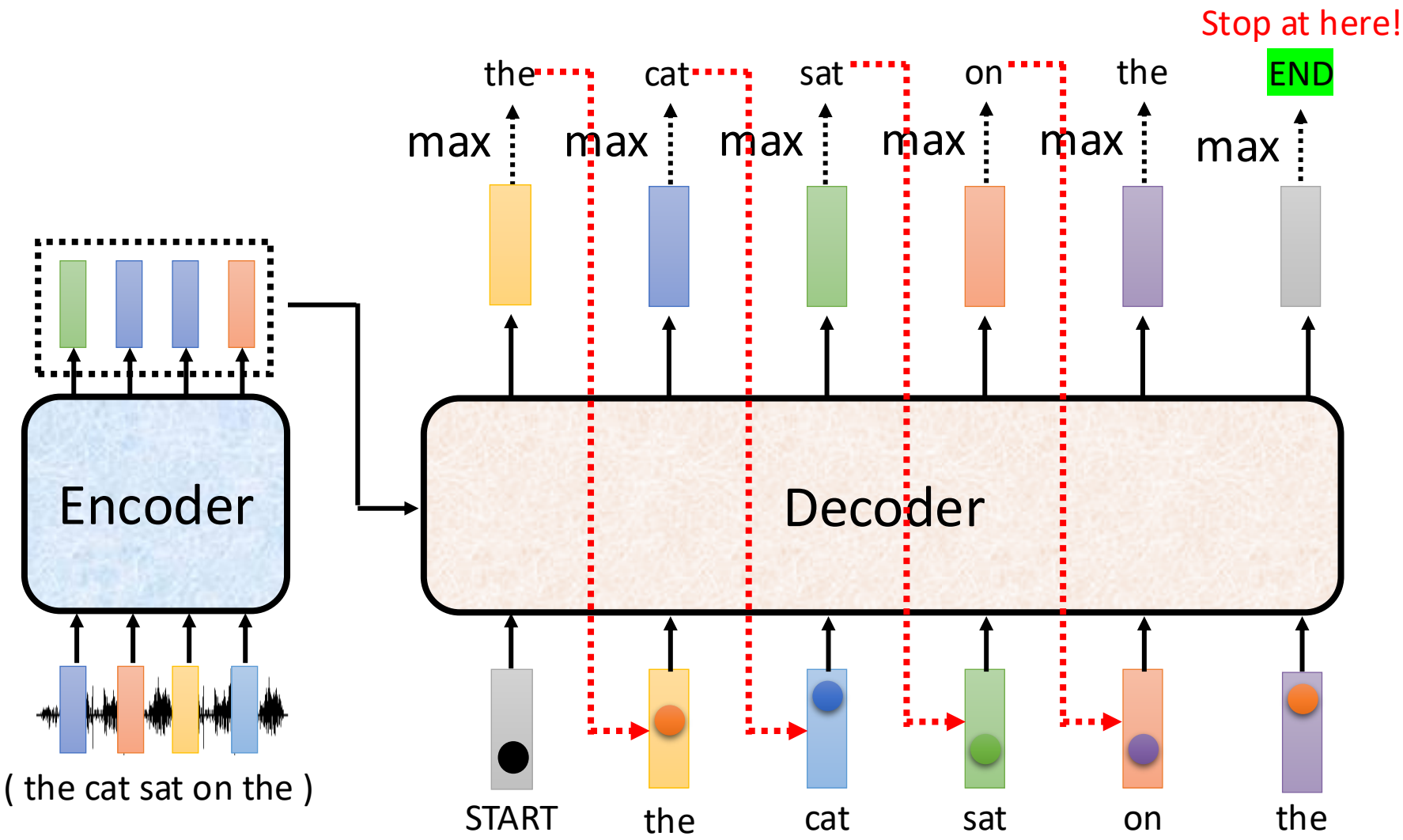
Recap: Original Transformer

https://arxiv.org/abs/1706.03762





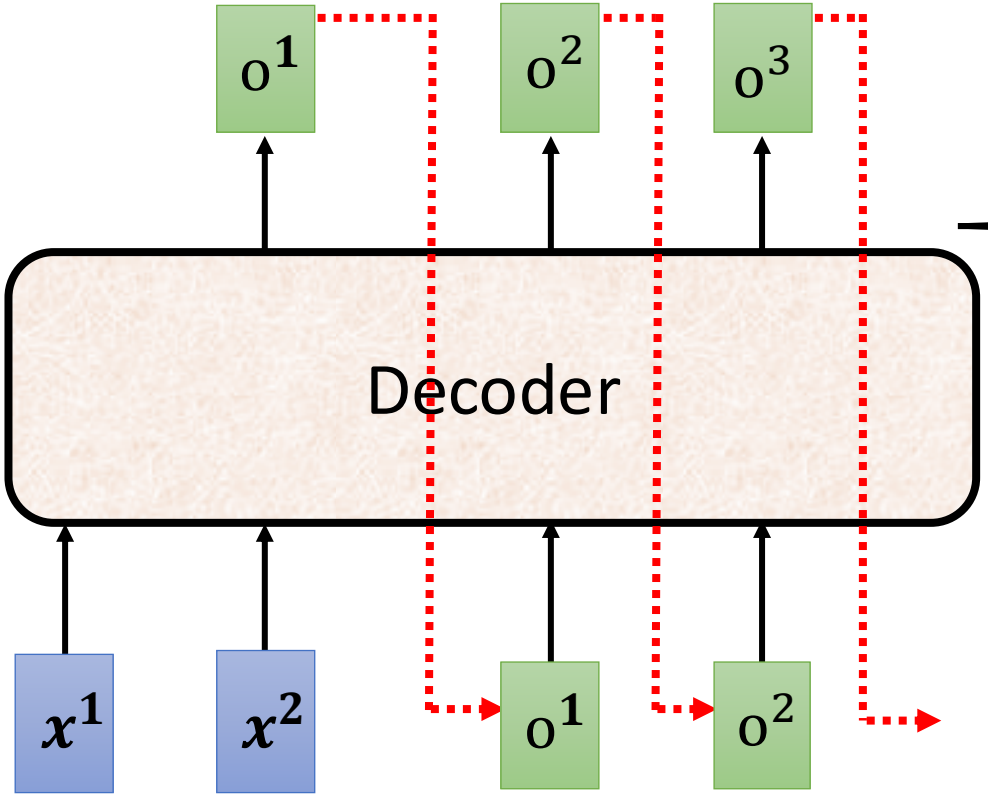
Decoder's Autoregressive Inference

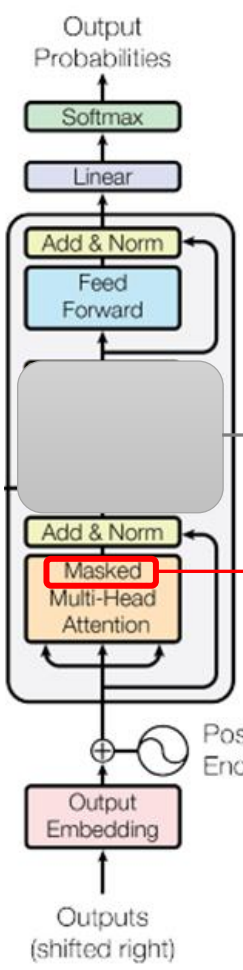


credit: Prof. Hung-yi Lee,

Transformer's Decoder

Decoder-only





no cross-attention (no encoder)

masked self-attention

GPT: learned position embeddings

credit: Prof. Hung-yi Lee,

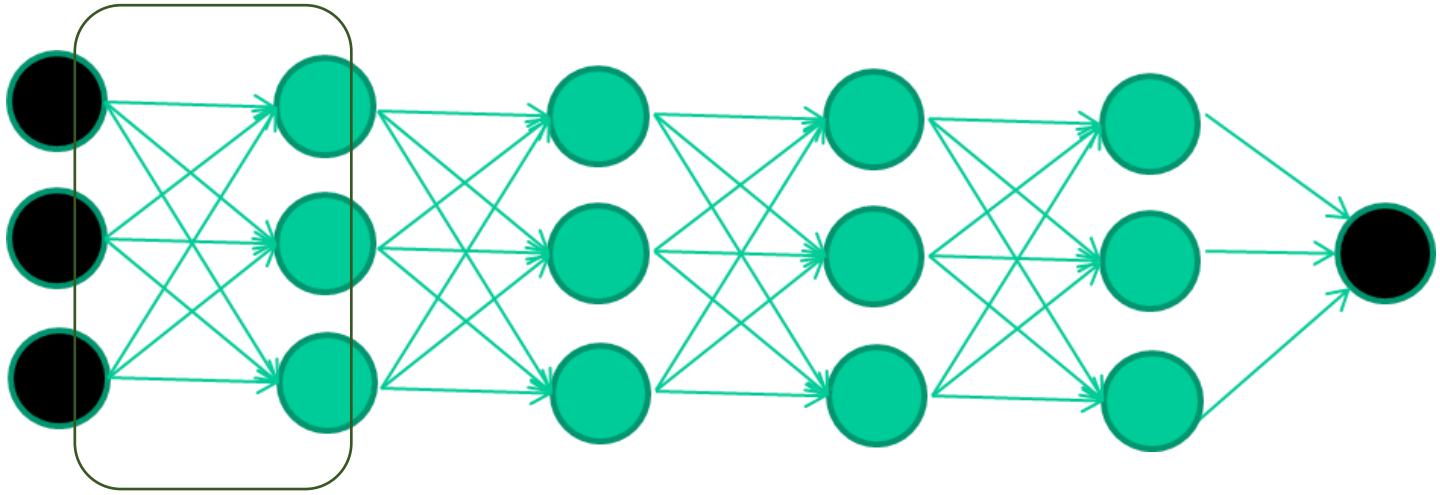
Recap : Transformer architecture

- Recap (word vectors, attention, bag-of-words)
- Self-attention Basics (Q, K, V, scaling, attention map)
- Transformer Block Basics (self-attn + heads, positions, residual + norm)
- Masks & Model Families (encoder-only, encoder-decoder, decoder-only)

Roadmap : Today

- **Training Stages (encoder-only, decoder-only)**
- Training tactics (tokens, next-token loss, Adam)
- Pretraining and Pretrained GPT models (GPT-1/2/3)
- Post-training (SFT, aligning a GPT: RLHF, DPO)

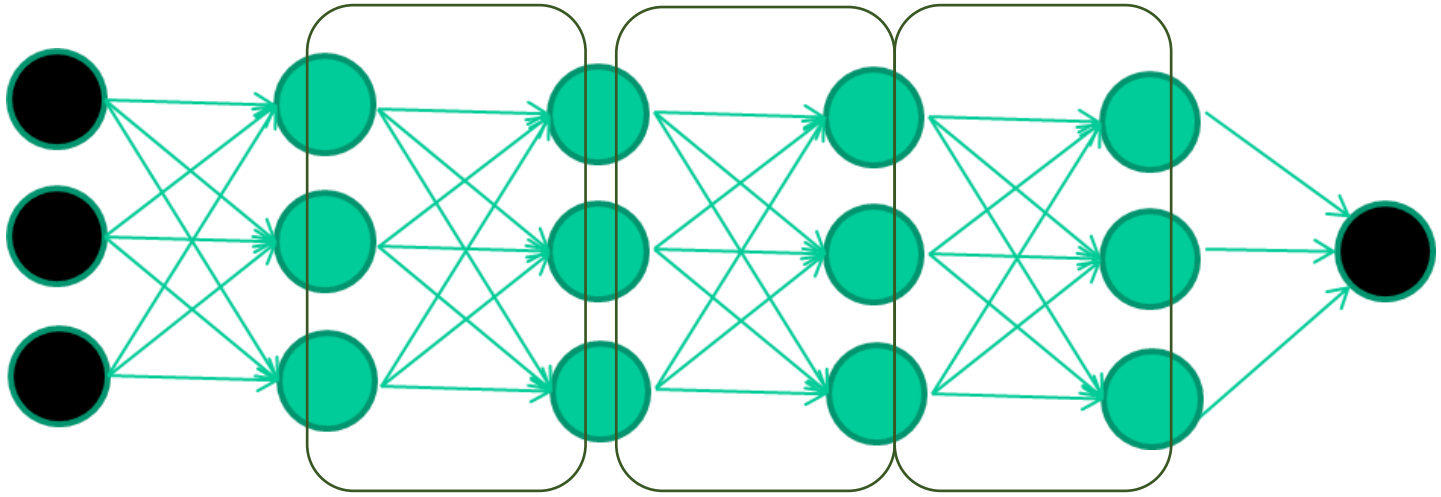
New Ways to Train multi-layer DNNs…



Train **this** layer first

https://www.macs.hw.ac.uk/~dwcorne/Teaching/introdl.ppt

The new way to train multi-layer DNNs…



Each layer can be trained well first with some types of self-supervised loss (e.g. reconstruction loss)

Basically, it is forced to learn good features that describe what comes from the previous layer

https://www.macs.hw.ac.uk/~dwcorne/Teaching/introdl.ppt

Two Main: Unsupervised Pre-training vs. SFT

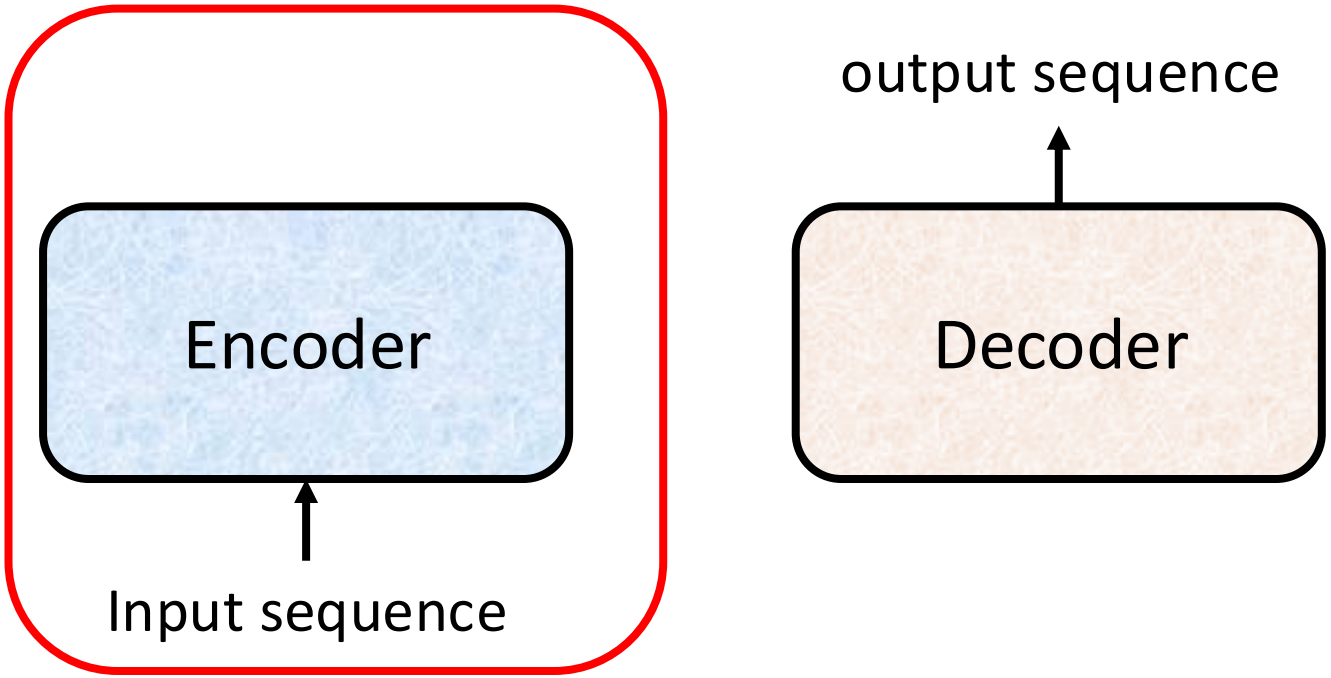
1. Unsupervised Pre-training
   - Use **unlabeled** data
   - Work bottom-up
     - Train hidden layer 1. Then fix its parameters.
     - Train hidden layer 2. Then fix its parameters.
     - …
     - Train hidden layer n. Then fix its parameters.
2. Supervised Fine-tuning (SFT)
   - Use **labeled** data to train the DNN that finished by the above pretraining
   - Refine the features by back-propagation so that they become tuned to the end-task

Pre-Training of DNN layer-layer Benefits:

- Layer-wise pre-training can help with:
  - Learn features at different levels of abstraction
  - DNN Model Training can be tricky due to Nonconvexity
  - DNN Model Training can be tricky due to Vanishing gradients

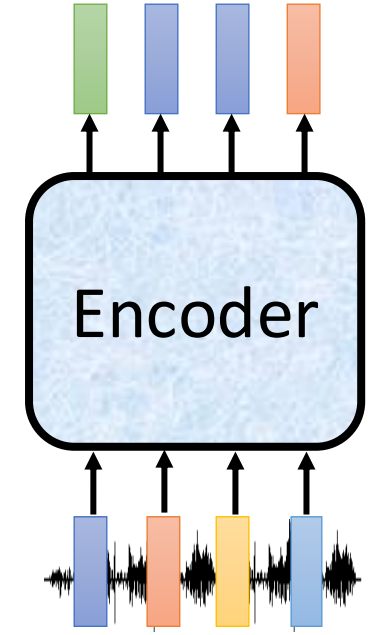
- Many different forms of pre-training ,
  - Mostly on unlabeled data, like web text
  - for instance,
    - Word embedding layers via skip-gram …
    - Pretrained GPT models via next-token LM training
    - Pretrained BERT models via masked LM training

Training

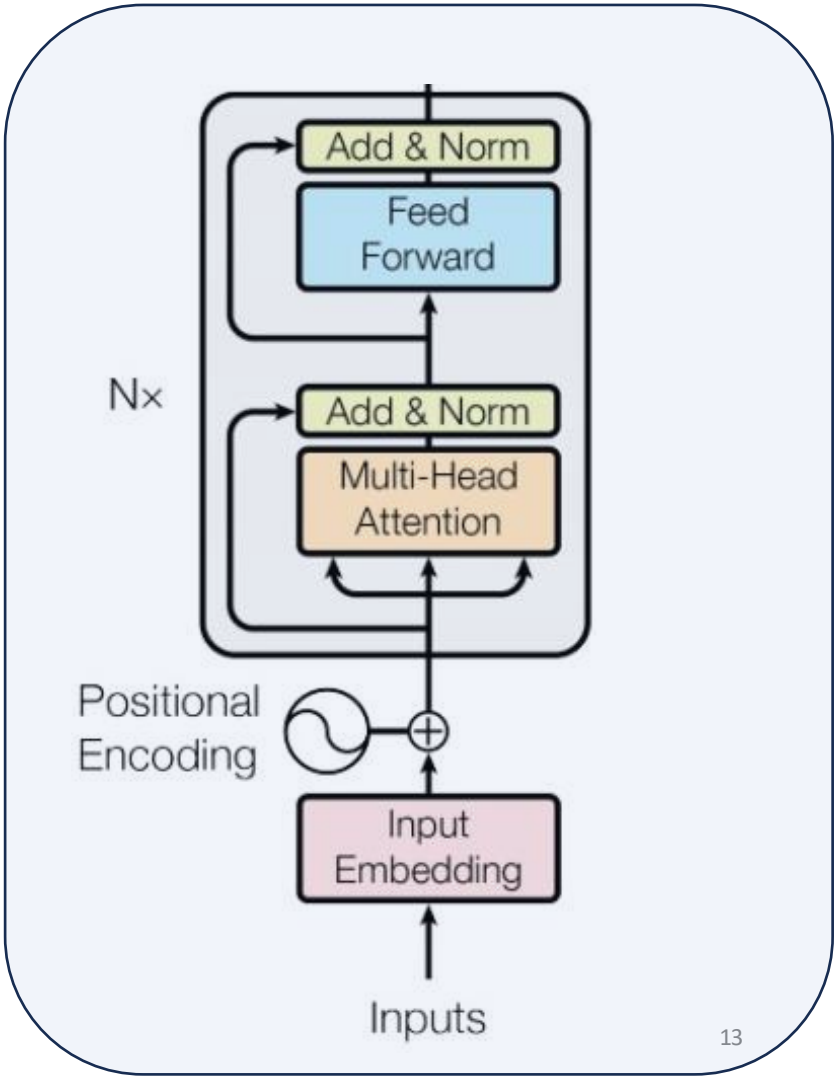


Transformer's ***Encoder Only, e.g., BERT***

Recap:

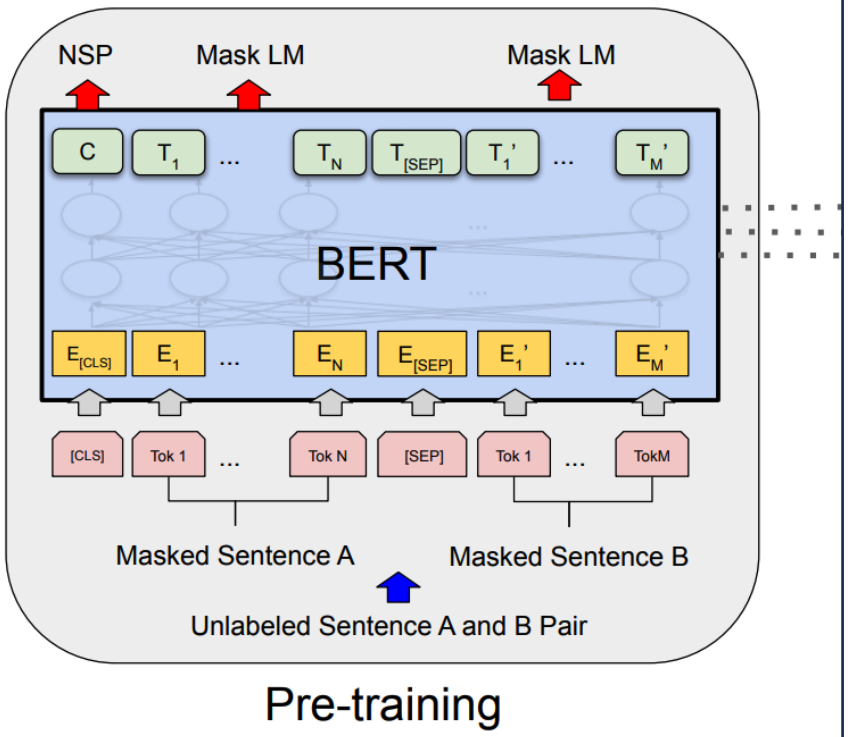


(the cat sat on the)



BERT Model Family

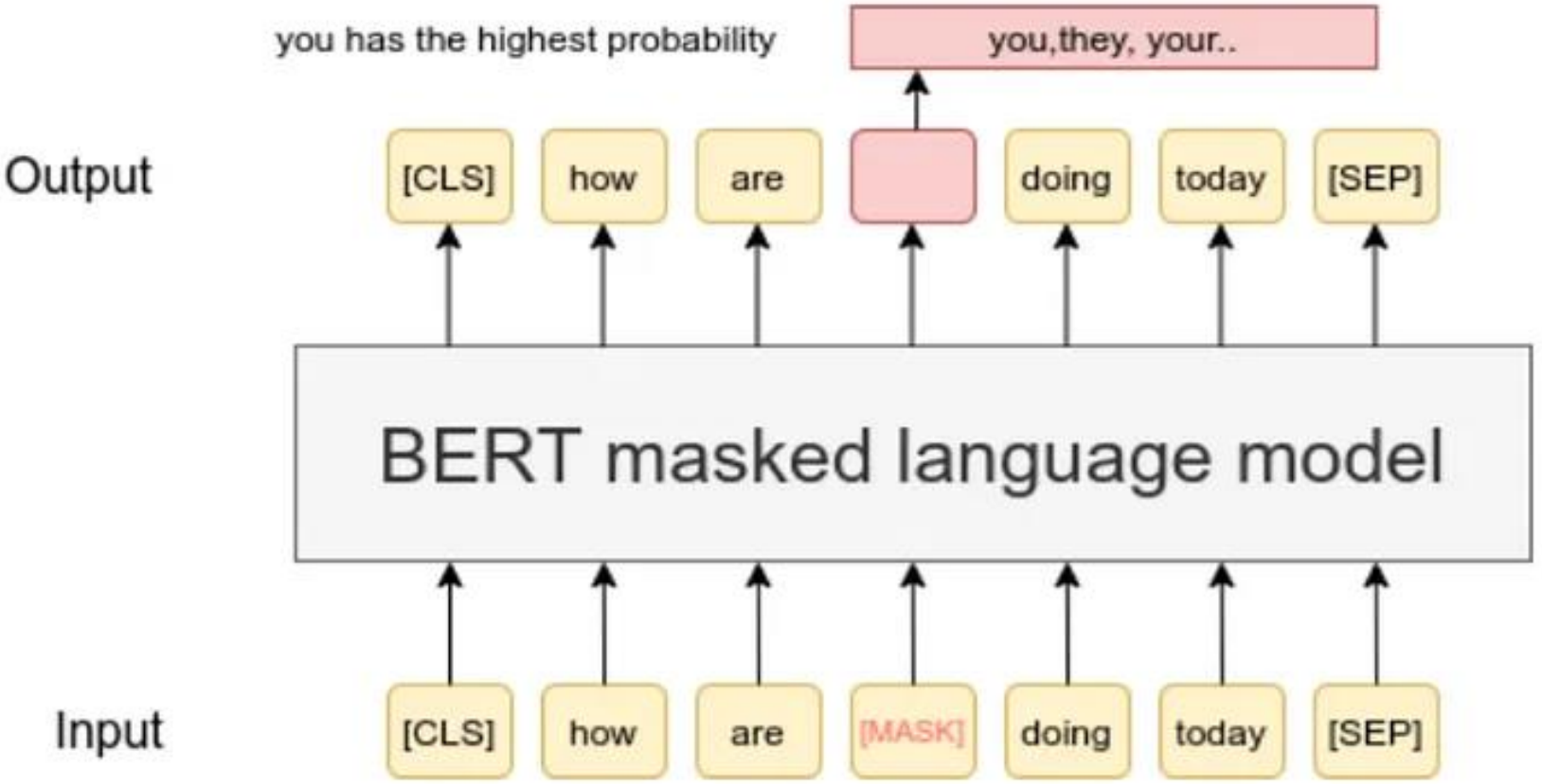
- 1. Pre-train Intuition: representation should be robust to the introduction of noise
  - Masked Language Model (MLM)
  - [MASK]: a unique token introduced in the training process to mask some tokens
  - Predict masked tokens based on their context information,



Pre-training

<- Using Large amount of unlabeled txt corpus

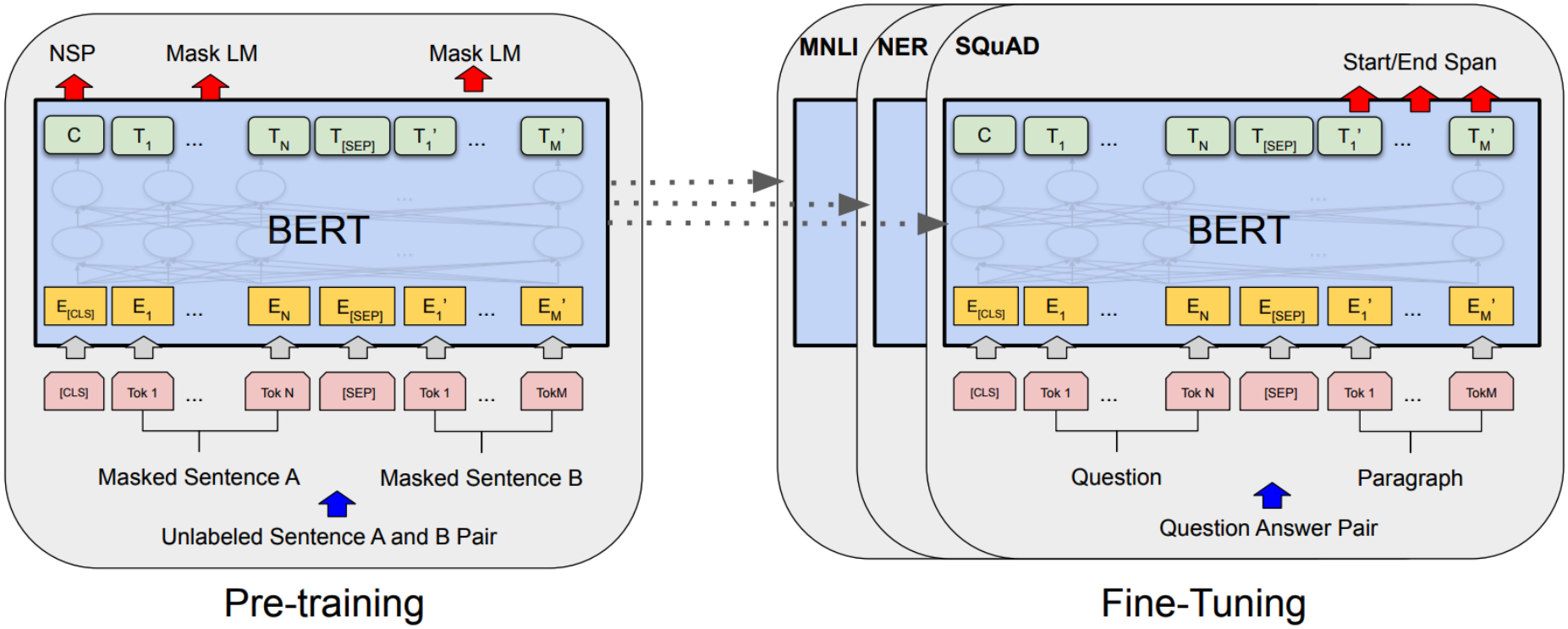
Predict masked tokens based on their left + Right context,



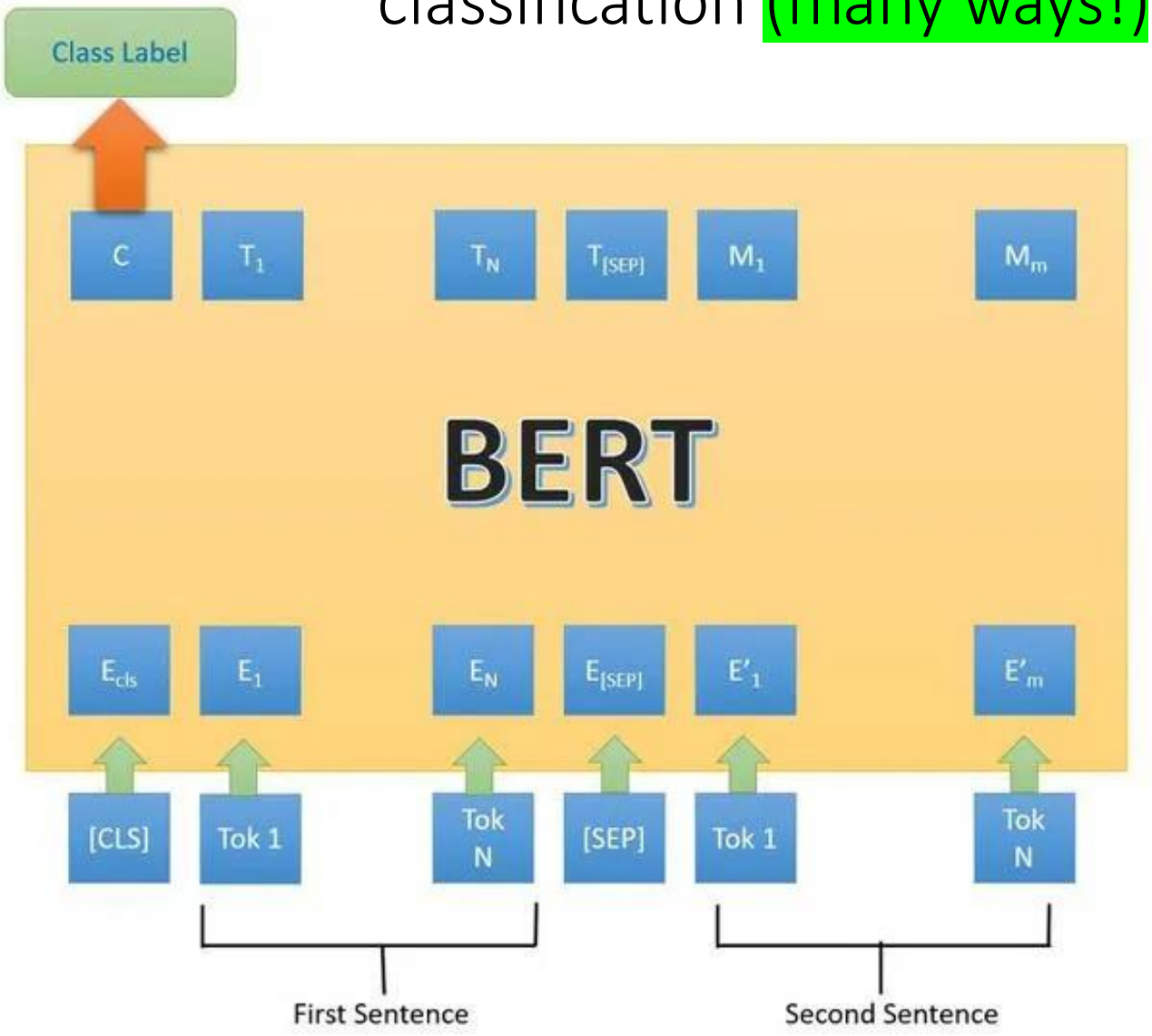
https://editor.analyticsvidhya.com/uploads/13216BERT_MLM.png

BERT Model Family

- 2. Fine Tuning to downstream tasks
  - With Tasks like Questions-Answering Sentence Pairs; Name Entity Recognition Tagging; …..
  - Downstream tasks' label provide fine tuning training signals.



How BERT adapts to sentence classification (many ways!)



https://editor.analyticsvidhya.com/uploads/13216BERT_MLM.png

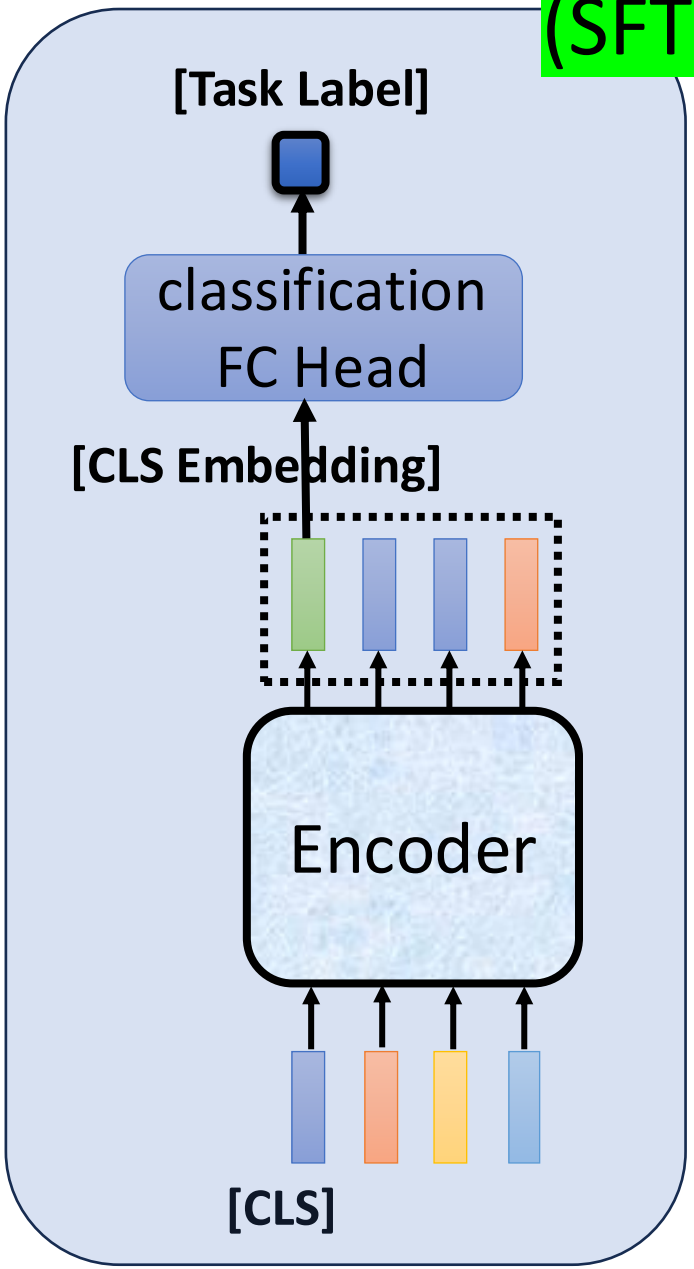
How BERT adapts to sentence classification (many ways)

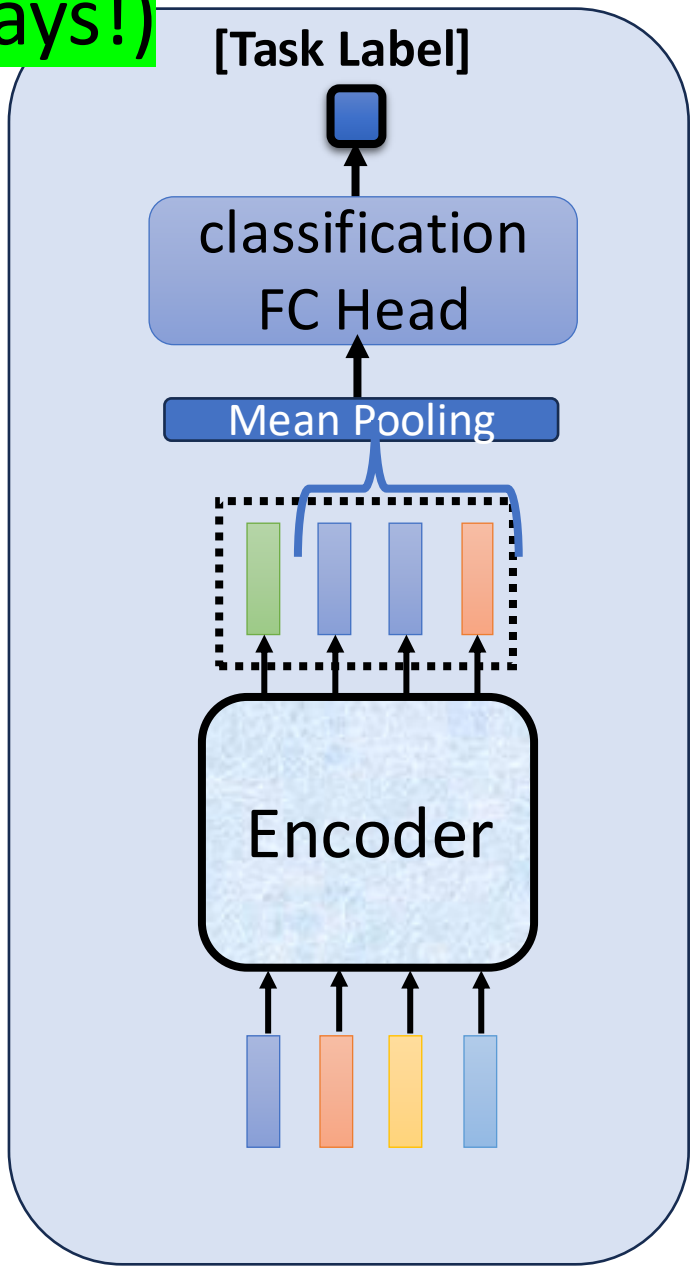
A. First Way: [CLS] token.

1. BERT puts a special `[CLS]` token at the start of every input: `[CLS] the movie was good [SEP]; and`
2. **Use [CLS]'s output vector.** After the encoder layers, the final vector at the [CLS] position represents the whole sequence. With full, unmasked self-attention, that position can attend to every token.
3. **Add a small classifier head.** One linear layer (D → number of classes) and a softmax.
4. **Fine-tune everything.** Train the head and all the BERT weights together with cross-entropy on the labelled task, usually for only a few epochs.

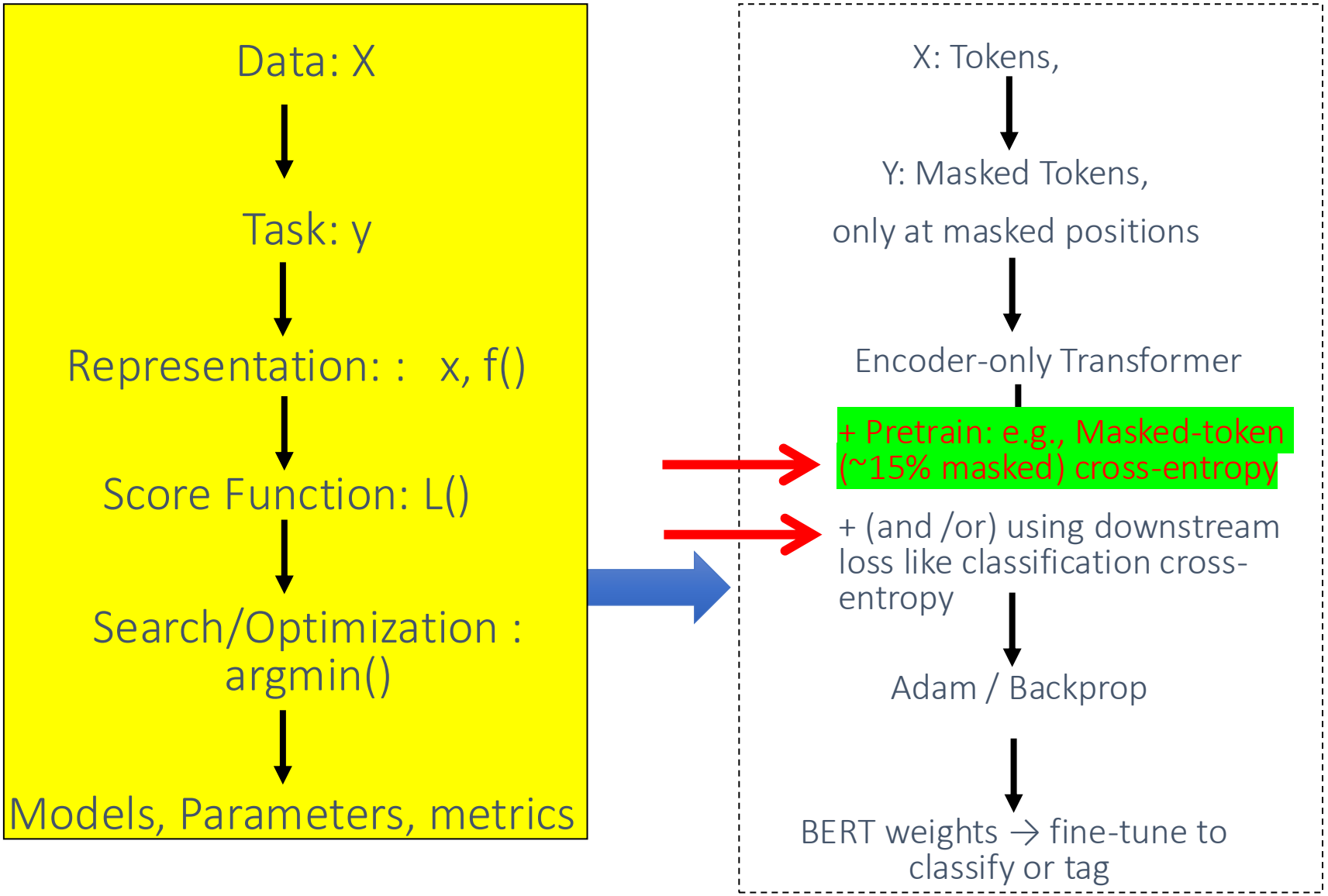
B. Mean pooling instead of [CLS]: average the output vectors over the real tokens, skipping pads. Common for sentence embeddings (Sentence-BERT) and for simple classifiers trained from scratch. With padding, it must be a masked mean: `(h * mask).sum(1) / mask.sum(1)`, which uses the padding mask.

(SFT many ways!)





Summary: Encoder-only Transformer MLM's Training

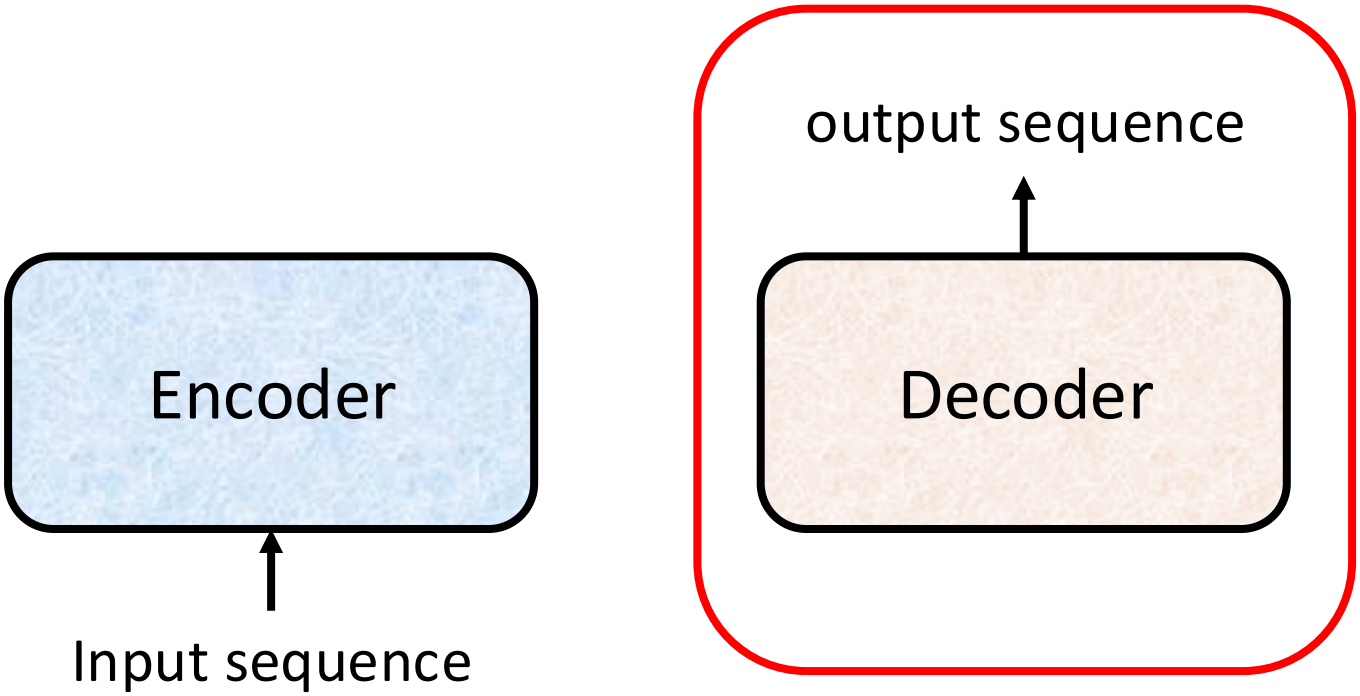


Transformer Encoder Training Nutshell:

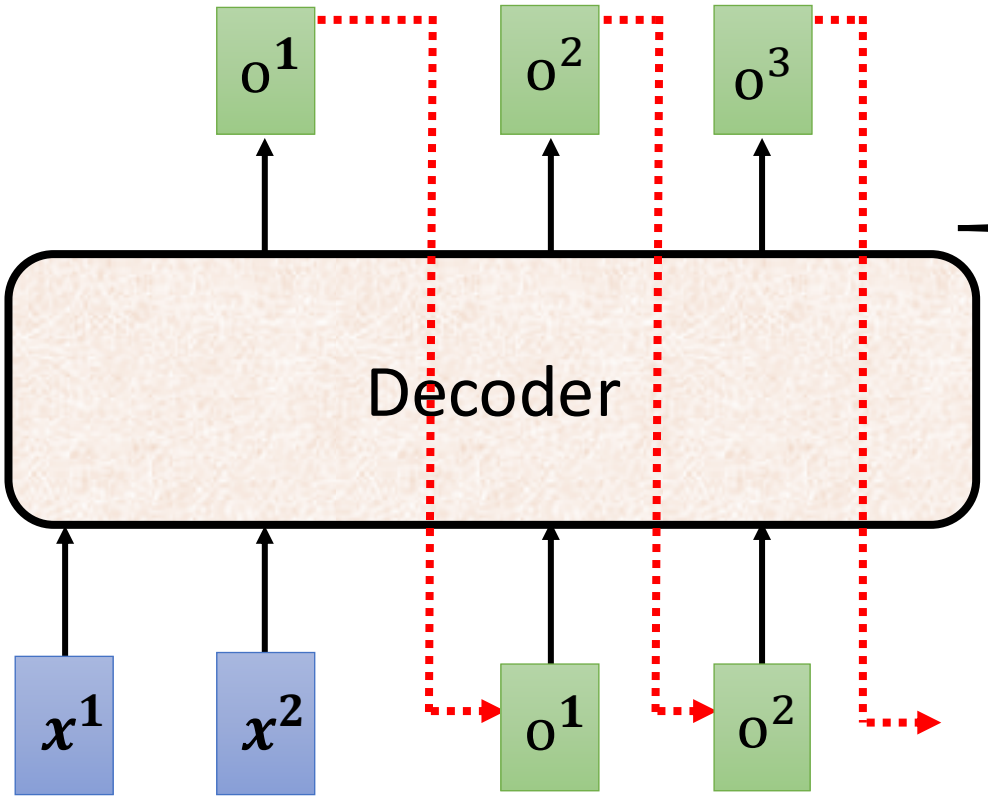
- **Contrast with GPT:** no causal mask, a loss only at masked positions, fine-tuning adds a task head, and inference is one forward pass, with no generation loop.
- Source: Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL 2019

Row	① Pretraining (masked LM)	② Fine-tuning
X	unlabeled text: BooksCorpus (800M words) + Wikipedia (2.5B); WordPiece tokens; sentence pairs A, B	labeled task data: sentences or pairs + labels
Y	the 15% picked tokens (80% [MASK], 10% random, 10% kept); + IsNext / NotNext (50/50)	task label: a class from [CLS], a tag per token, or an answer span
f()	encoder-only Transformer, bidirectional (no causal mask)	same encoder + a small task head
L()	cross-entropy at the picked positions only (+ next-sentence cross-entropy)	task cross-entropy (classify, tag, or span)
Search	Adam, peak LR 1e-4, 10k warm-up steps + linear decay; 1M steps × 256 sequences	Adam on all weights, LR 2e-5 to 5e-5, batch 16/32, 2–4 epochs
Output	BERT weights: contextual token embeddings; MLM accuracy	task model; accuracy / F1

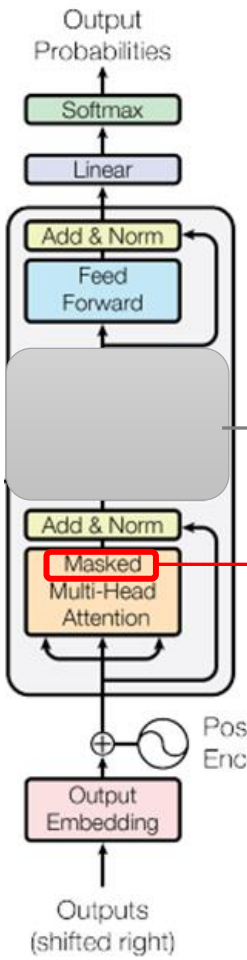
Training



Recap: Decoder-only



Transformer's Decoder



no cross-attention (no encoder)

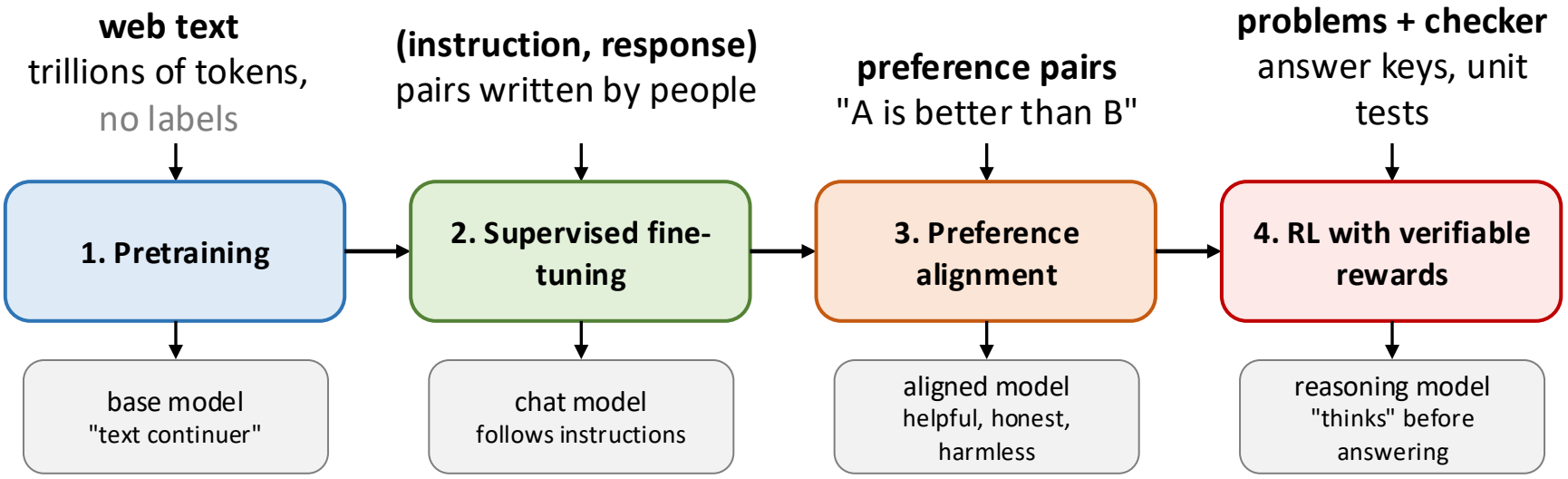
masked self-attention

GPT: learned position embeddings

credit: Prof. Hung-yi Lee,

How a base decoder model becomes a chat/reason agent's model

the training pipeline

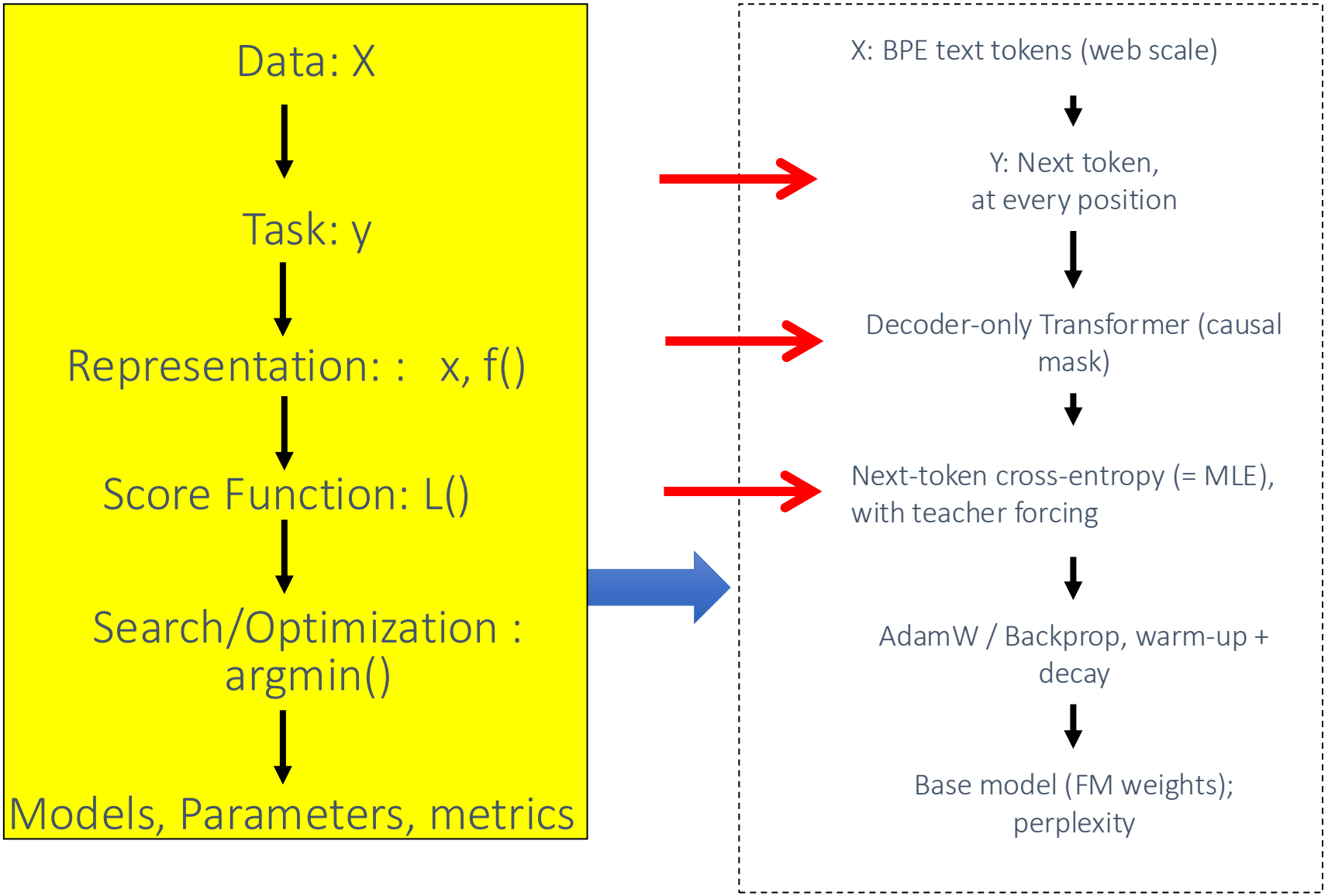


- For each stage we ask the course's four questions: **Data? Loss? Optimization? What is learned?**

- Not a training stage: **prompting / in-context learning**, which changes the *input*, not the weights.

credit: Peng Wang, UVA CS 4774 L15

① Pretraining: the decoder-only

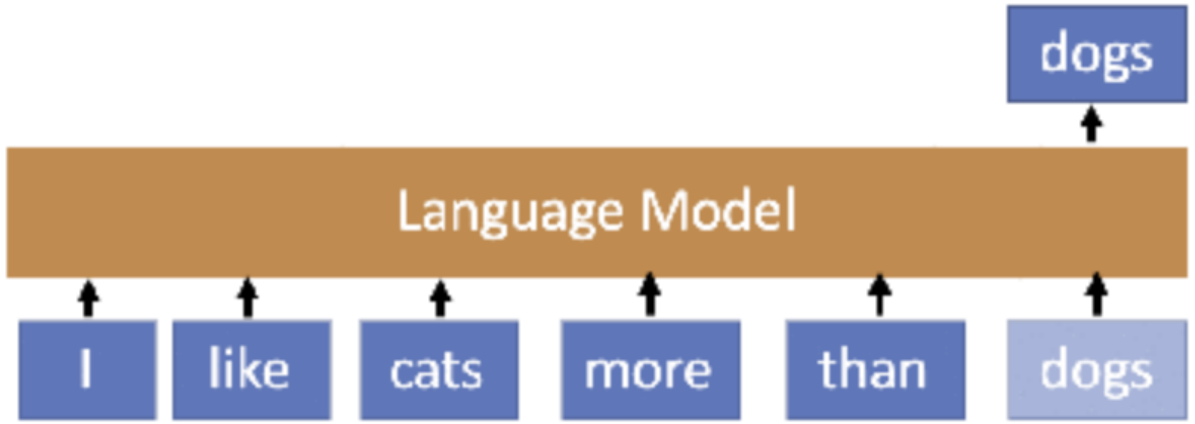


A language model assigns probabilities to text

- A **language model (LM)** assigns a probability to any sequence of tokens $x_1, \ldots, x_T$.
- Chain rule of probability:

$$P(x_1, \ldots, x_T) = \prod_{t=1..T} P(x_t \mid x_{<t})$$

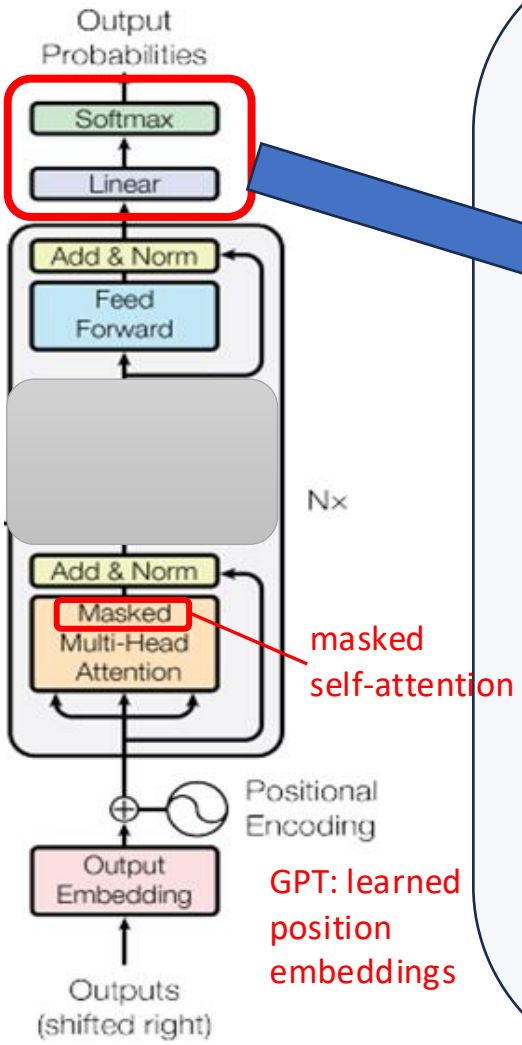
- Example: $P(\text{I eat the apple}) = P(\text{I}) \cdot P(\text{eat} \mid \text{I}) \cdot P(\text{the} \mid \text{I eat}) \cdot P(\text{apple} \mid \text{I eat the})$

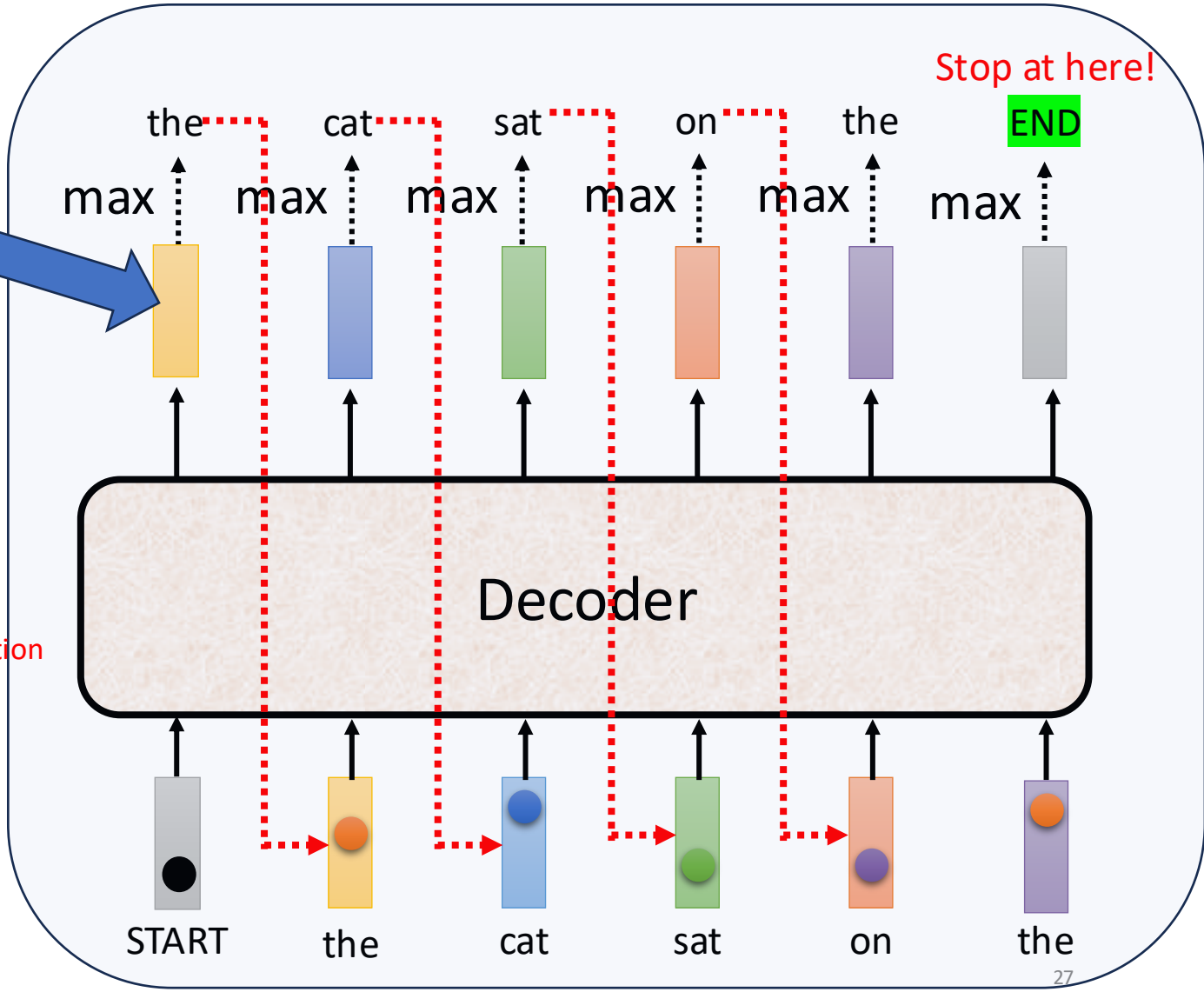


The prediction scheme for a traditional language model. Shaded words are provided as input to the model while unshaded words are masked out.

Decoder's Autoregressive Inference

$$P(x;\theta) = \prod_{n=1}^{N} P(x_n | x_{<n}; \theta)$$



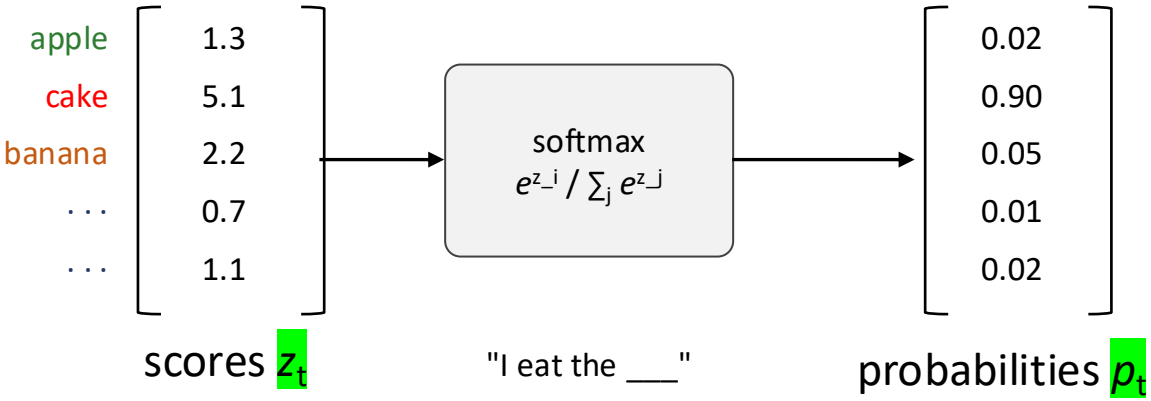


credit: Prof. Hung-yi Lee,

The output layer is multi-class logistic regression

At position t the decoder gives $h_t \in \mathbb{R}^d$. Then

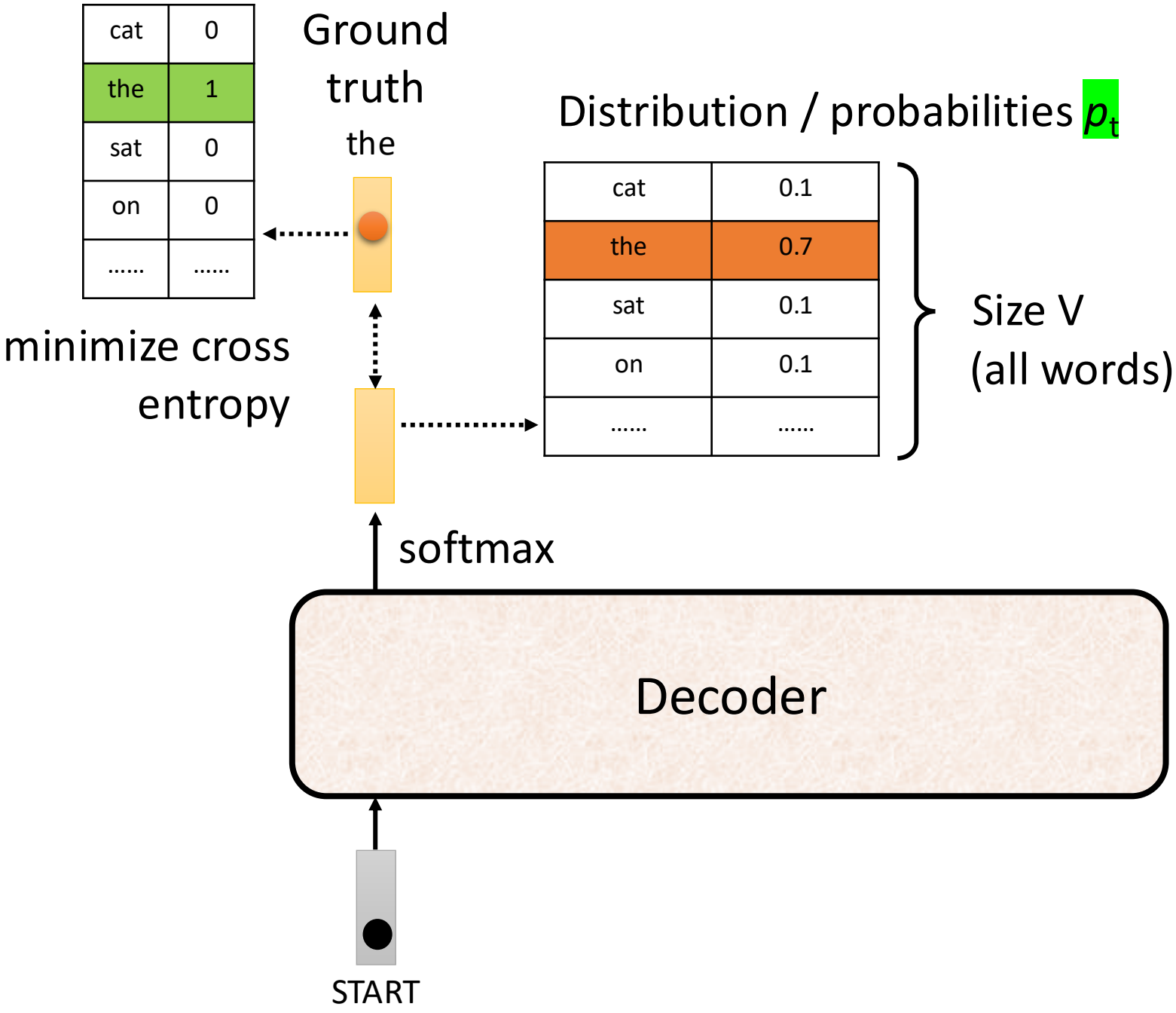
$$z_t = W_{out}\, h_t + b \in \mathbb{R}^{|V|}, \qquad p_t(i) = \exp(z_{t,i}) / \sum_{j=1..|V|} \exp(z_{t,j})$$



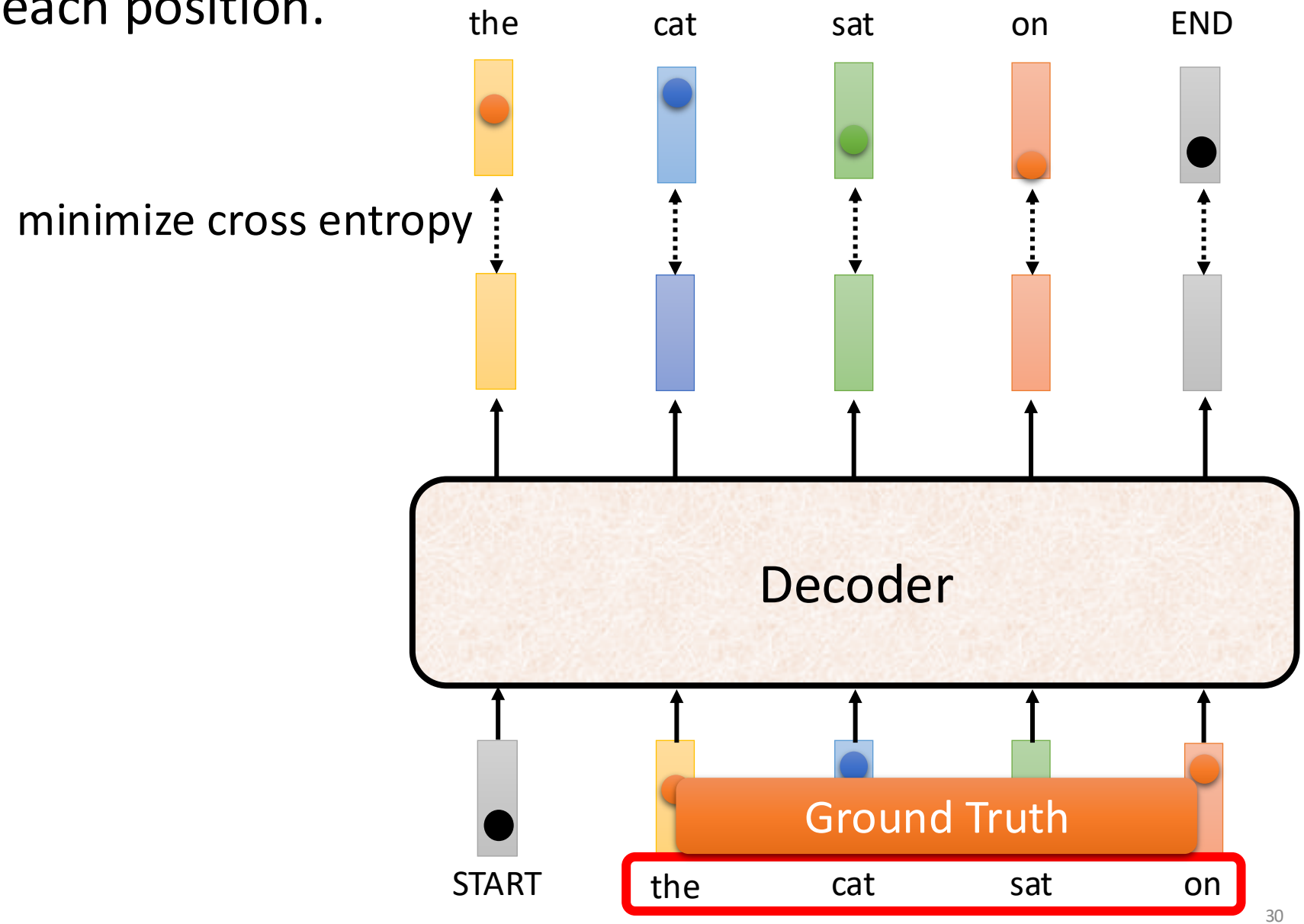
logits: one score per token

You've seen this (logistic regression): $p(y=1 \mid x) = \sigma(w^\top x + b)$ is the 2-class case; softmax is its K-class generalization. Here the features are h_t and $K = |V| \approx 100{,}000$.

credit: Peng Wang, UVA CS 4774 L15


cat 0
the 1
sat 0
on 0
....
Ground truth
the
Distribution / probabilities p_t
cat 0.1
the 0.7
sat 0.1
on 0.1
....
Size V (all words)
minimize cross entropy
softmax
Decoder
START

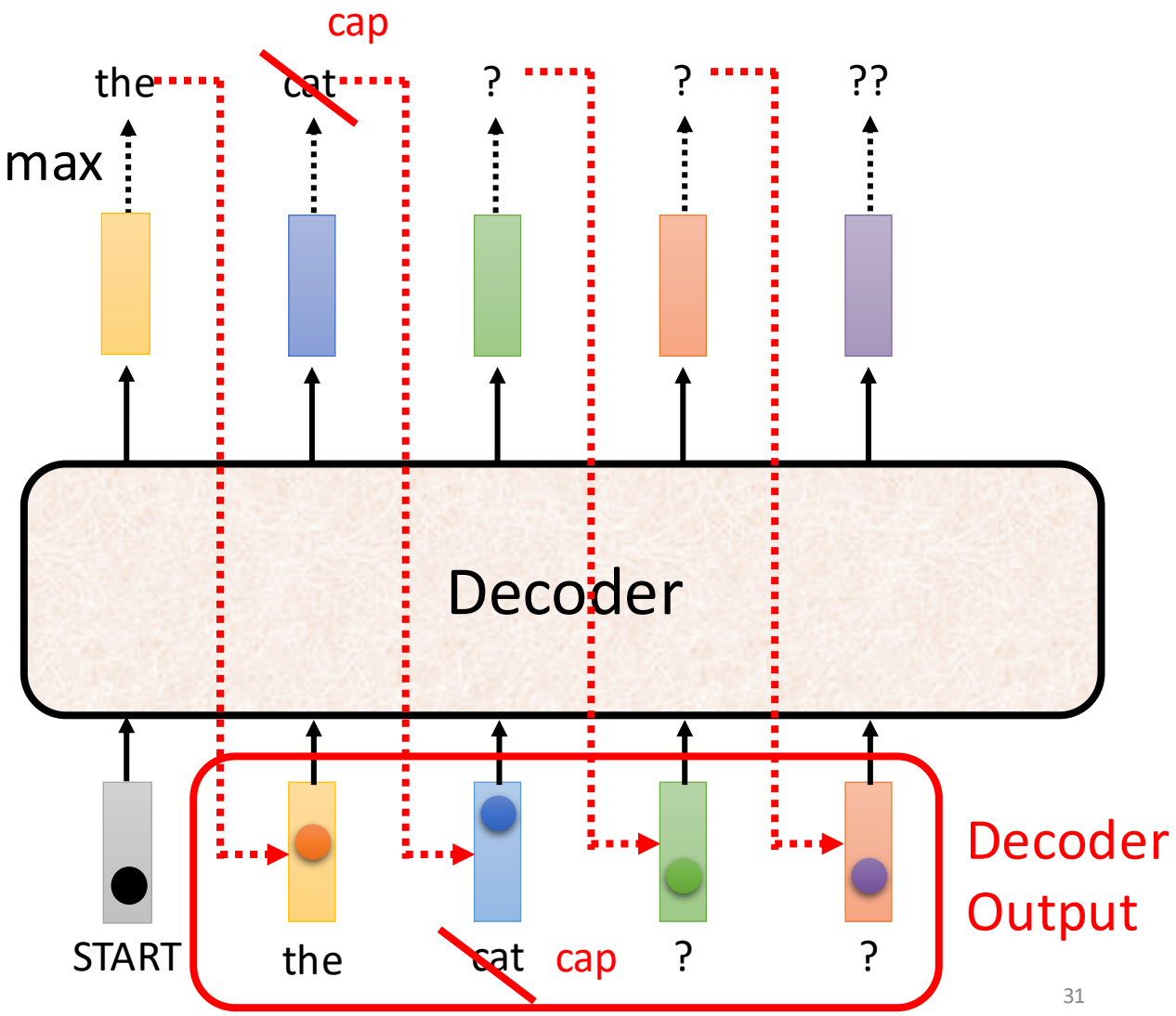
Teacher Forcing: using the ground truth as input for each position.



If without teacher forcing!~

Each step takes everything produced so far and predicts one more token

➔ Decoder's Autoregressive (Forward Step!)

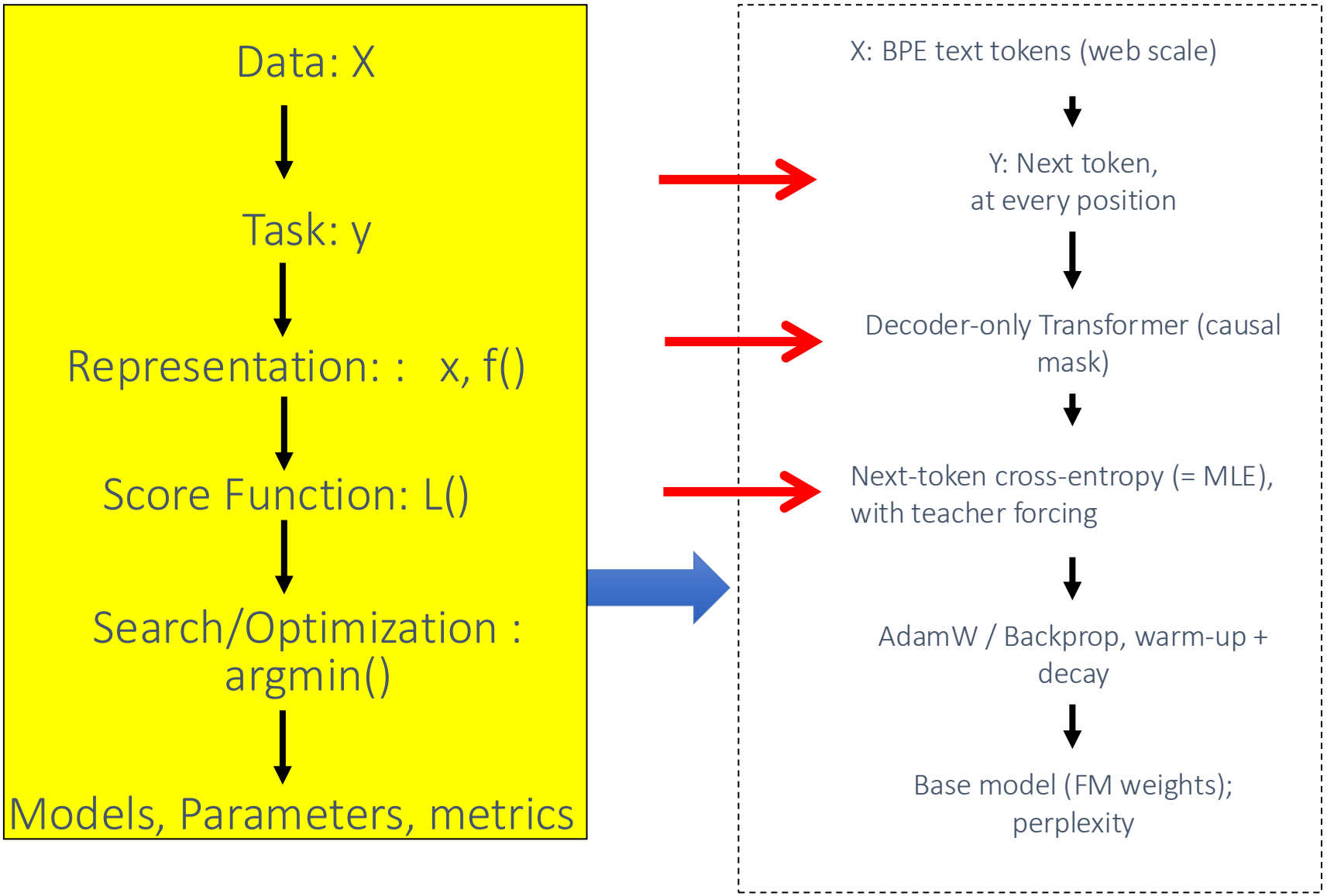


credit: Prof. Hung-yi Lee,

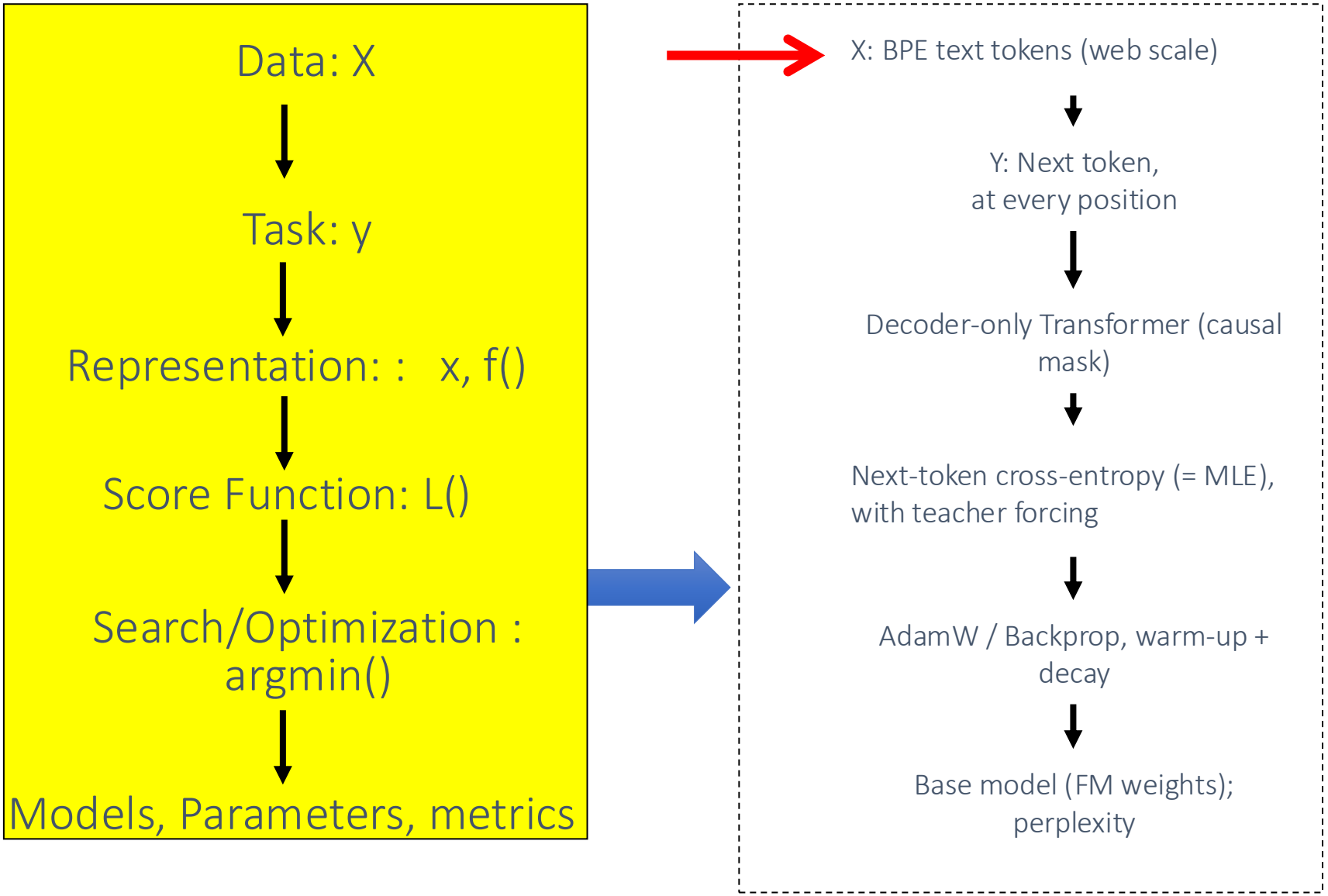
Roadmap : Today

- Training Stages (encoder-only, decoder-only)
- **Training tactics (tokens, next-token loss, Adam)**
- Pretraining and Pretrained GPT models (GPT-1/2/3)
- Post-training (SFT, aligning a GPT: RLHF, DPO)

① Pretraining: the decoder-only



① Pretraining: the decoder-only

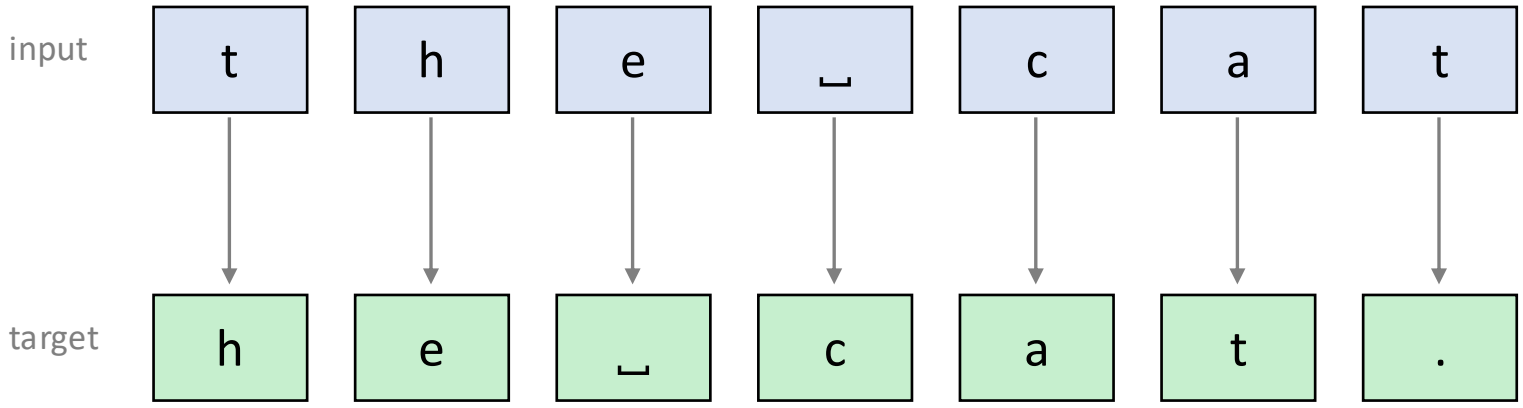


Tokens: characters, words, or subwords

	Characters	**Words**	**Subwords (BPE)**
“unhappiness”	u n h a p p i n e s s (11)	unhappiness (1) or <unk>	un · happi · ness (3)
Vocabulary size	tiny (tens)	huge (100K+)	≈ 32K – 200K
Unknown words	never	often	never (falls back to bytes)
Sequence length	long	short	medium

- **Byte-pair encoding (BPE)**: start from characters, repeatedly merge the most frequent adjacent pair
  - l o w e r → lo w e r → low e r → low er → …
- GPT-2/3/4, Llama and Claude all use subword tokenizers

Summary: predict the next token



- Targets = inputs shifted left by one; with the causal mask, position t sees inputs 0 … t only
- Loss = average cross-entropy over all real positions (<pad> ignored)
- Teacher forcing: the model always reads the **true** previous characters during training
- Thanks to the causal mask, all T predictions train in parallel in one forward pass
- Self-supervised: raw text is its own label

Why does next-word prediction teach so much?

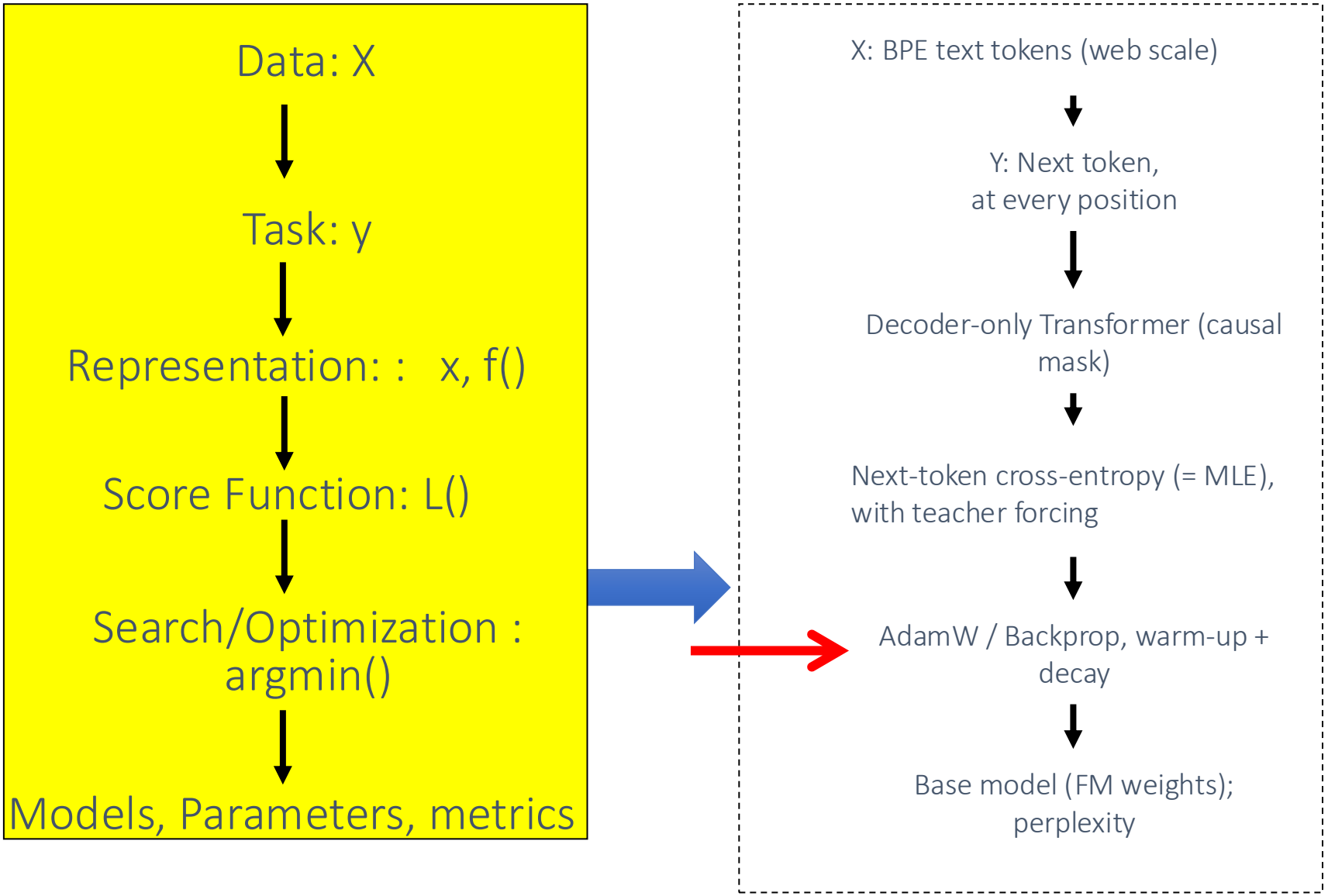
Minimizing one loss on trillions of words forces the model to pick up every regularity that helps predict the next token:

to fill the blank it must learn . . .	example
grammar	In my free time, I like to {run, banana}
lexical semantics	At the zoo we saw giraffes, lions, and {zebras, spoon}
world knowledge	The capital of Denmark is {Copenhagen, London}
sentiment	Edge of my seat the whole time. The movie was {good, bad}
translation	The word for "pretty" in Spanish is {bonita, hola}
arithmetic	3 + 8 + 4 = {15, 11}

Pretraining is implicit multi-task learning. Nobody wrote a sentiment loss or a translation loss; the next-token loss already contains all of them.

credit: Peng Wang, UVA CS 4774 L15

① Pretraining: the decoder-only

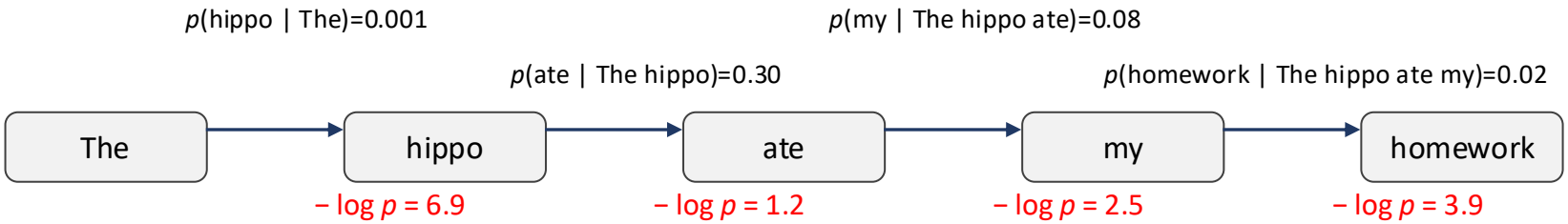


Training with cross-entropy = maximum likelihood

on the corpus

Pretraining loss (GPT-style, "causal" language modeling):

$$\mathcal{L}(\theta) = -(1/N) \sum_{t=1..N} \log p_\theta(x_t \mid x_{<t})$$



- Each probability is conditioned on the *whole prefix* $x_{<t}$, not just the previous word.
- Target at each position = the **actual next token**; loss = its NLL; average over positions.
- **Self-supervised**: the labels are free, the next word is already in the text.

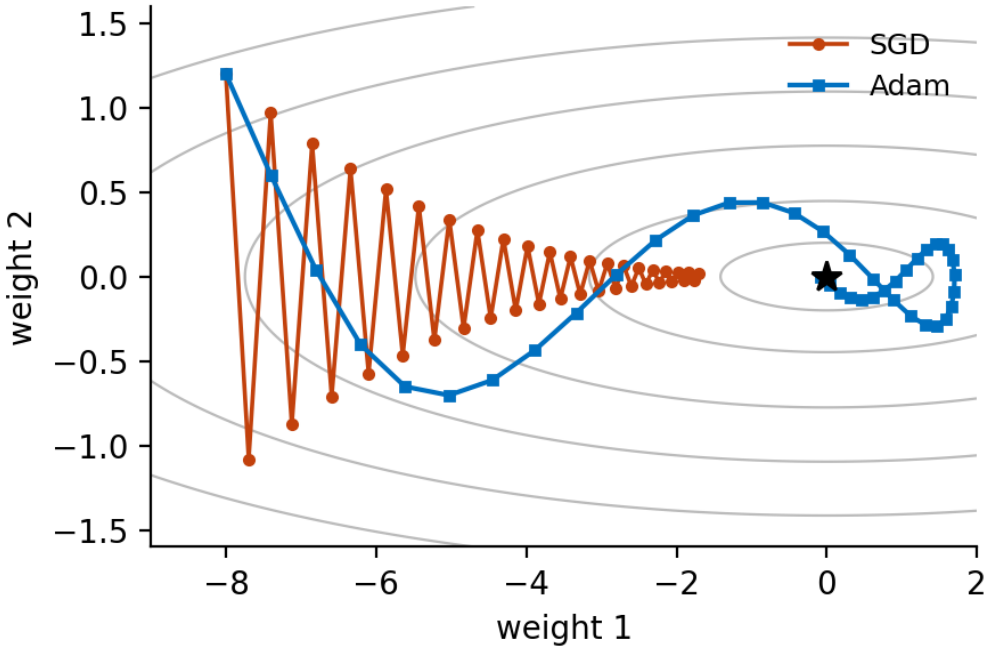
You've seen this (MLE): $\max \prod_t p_\theta(x_t \mid x_{<t}) \Leftrightarrow \min -\sum_t \log p_\theta(x_t \mid x_{<t})$.
Pretraining with next token-predict *is* maximum likelihood !!!

credit: Peng Wang, UVA CS 4774 L15

Optimizers: one step (learning rate) size is not enough

$$\theta_t = \theta_{t-1} - \gamma\, g_t$$

One learning rate γ for every weight (parameter, dimen, et al);
g_t = minibatch gradient



- Steep weight 2 caps γ: SGD diverges above γ = 0.04
- The moving crawls along the flat weight 1 direction
- Same start, 40 steps, best γ tried:
  - SGD: loss 68 → 1.45
  - Adam: loss 68 → 0.003

This loss contour map: steep in weight 2, flat in weight 1

Adam: two running averages

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$

Momentum: average direction

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2)\, g_t^2$$

Scale: average squared size

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}$$

Bias fix: both start at 0

$$\theta_t = \theta_{t-1} - \gamma \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}$$

Update: per-weight step

- Everything is per weight; g_t^2 is element-wise
- PyTorch defaults: $\gamma = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Adam: momentum + a step size per weight

credit: Kingma & Ba, ICLR 2015; defaults from the PyTorch torch.optim.Adam documentation

Adam by hand: the first step

Two weights, first gradient g = (0.1, 10), $\gamma = 0.001$, defaults for β_1, β_2, ε

	Weight 1 (flat)	**Weight 2 (steep)**
Gradient g	0.1	10
$m_1 = 0.1\ g$	0.01	1
$s_1 = 0.001\ g^2$	0.00001	0.1
$\hat{m}_1 = m_1 / 0.1$	0.1	10
$\hat{s}_1 = s_1 / 0.001$	0.01	100
Adam step $\gamma\ \hat{m}_1 / \sqrt{\hat{s}_1}$	**0.001**	**0.001**
SGD step $\gamma\ g$	0.0001	0.01

Steep weight scaled down, flat weight scaled up: both move 0.001

Check: all gradients 10× larger. How far does each weight move?

AdamW: keep weight decay out of Adam

Only difference: where the weight decay λθ is added

Adam with L2 penalty

$$g_t = \nabla L(\theta_{t-1}) + \lambda \theta_{t-1}$$

$$\theta_t = \theta_{t-1} - \gamma \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}$$

λθ is divided by √ŝ: big-gradient weights barely decay

AdamW

$$g_t = \nabla L(\theta_{t-1})$$

$$\theta_t = \theta_{t-1} - \gamma \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon} - \gamma \lambda \theta_{t-1}$$

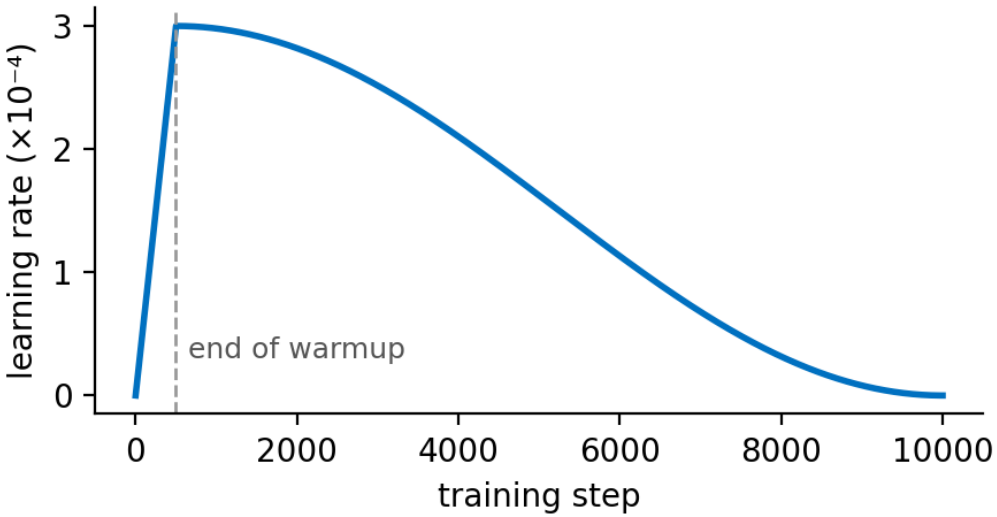
λθ skips m and s: every weight decays equally

```
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)
# weight_decay defaults: Adam 0, AdamW 0.01
```

AdamW is the usual choice for Transformers.

credit: Loshchilov & Hutter, ICLR 2019; defaults from the PyTorch optimizer documentation

Learning rate: warm up, then decay



- Warmup: start small while gradients are noisy and s is unsettled
- Decay: smaller steps near the end
- Example: 10,000 steps, 500 of warmup, peak 3×10^{-4}

```
def lr_at(t):              # called once per step, before opt.step()
    if t < W: return peak * t / W                              # linear warmup
    return peak * 0.5 * (1 + math.cos(math.pi * (t - W) / (N - W)))   # cosine
```

Debug with a fixed learning rate first; schedule later

credit: warmup as in Vaswani et al. 2017; cosine decay after Loshchilov & Hutter, ICLR 2017

Code: training loop and common bugs

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)    # PyTorch's default lr
loss_fn = nn.CrossEntropyLoss(ignore_index=0)          # 0 is <pad>
for epoch in range(n_epochs):
    for x, mask in loader:                             # x: (B, T)
        inp, tgt = x[:, :-1], x[:, 1:]                 # shift by one
        logits = model(inp, mask[:, :-1])              # (B, T-1, V)
        loss = loss_fn(logits.reshape(-1, V), tgt.reshape(-1))
        opt.zero_grad(); loss.backward(); opt.step()
```

Symptom	Cause	Fix
Shape error in the loss	logits have 3 axes, targets 2	reshape both, as above
Padding gets trained on	<pad> is a target	ignore_index=0
Odd next-token guesses	reading a pad position	use the last real position
Padding moves the class score	pads in the mean	(c * m).sum(1) / m.sum(1)
Later tokens change earlier logits	mask missing or after softmax	change inputs after t, compare

Adam: gradient descent with momentum and its own step size for each weight. Try a few learning rates and compare the loss curves.

Roadmap : Today

- Training Stages (encoder-only, decoder-only)
- Training tactics (tokens, next-token loss, Adam)
- **Pretraining and Pretrained GPT models (GPT-1/2/3)**
- Post-training (SFT, aligning a GPT: RLHF, DPO)

Scaling Laws for Neural Language Models

Jared Kaplan *
Johns Hopkins University, OpenAI
jaredk@jhu.edu

Sam McCandlish*
OpenAI
sam@openai.com

Tom Henighan
OpenAI
henighan@openai.com

Tom B. Brown
OpenAI
tom@openai.com

Benjamin Chess
OpenAI
bchess@openai.com

Rewon Child
OpenAI
rewon@openai.com

Scott Gray
OpenAI
scott@openai.com

Alec Radford
OpenAI
alec@openai.com

Jeffrey Wu
OpenAI
jeffwu@openai.com

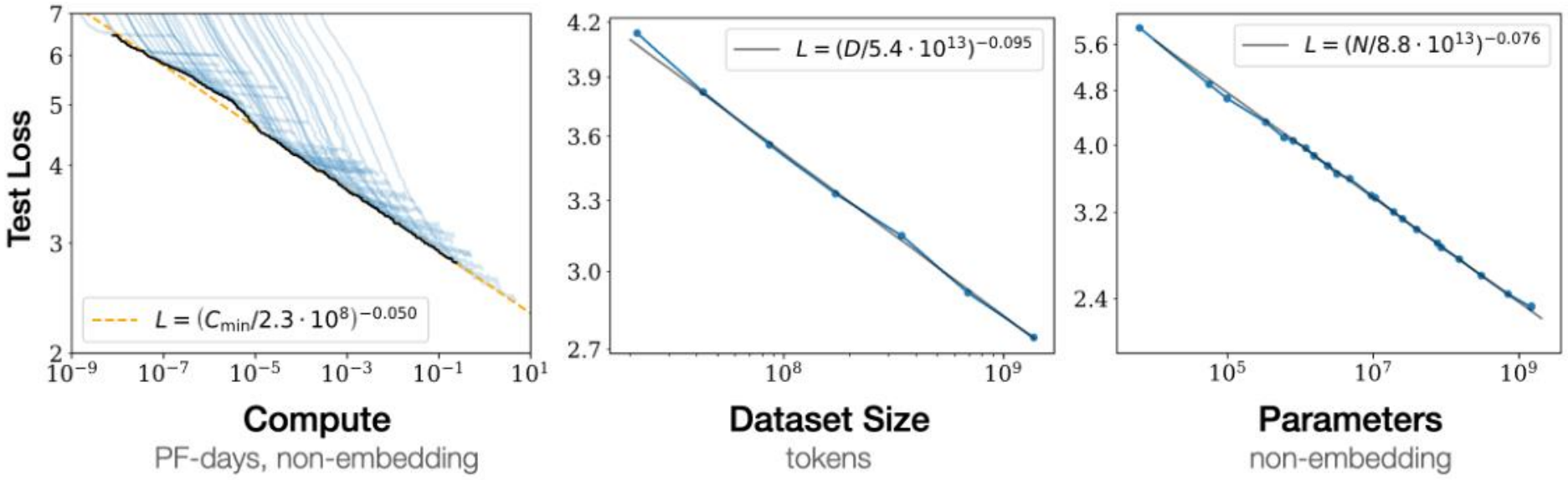
Dario Amodei
OpenAI
damodei@openai.com

[cs.LG] 23 Jan 2020

- **Data:** Llama-3 saw ~15 trillion tokens.
- **Parameters:** 0.1B (2018) → 175B (2020) → 405B (2024).
- **Compute:** thousands of GPUs for months.
- **Scaling laws** (Kaplan et al., 2020): test loss falls as a **power law** in each factor: straight lines on log-log axes.

Scale: more of everything, predictably better

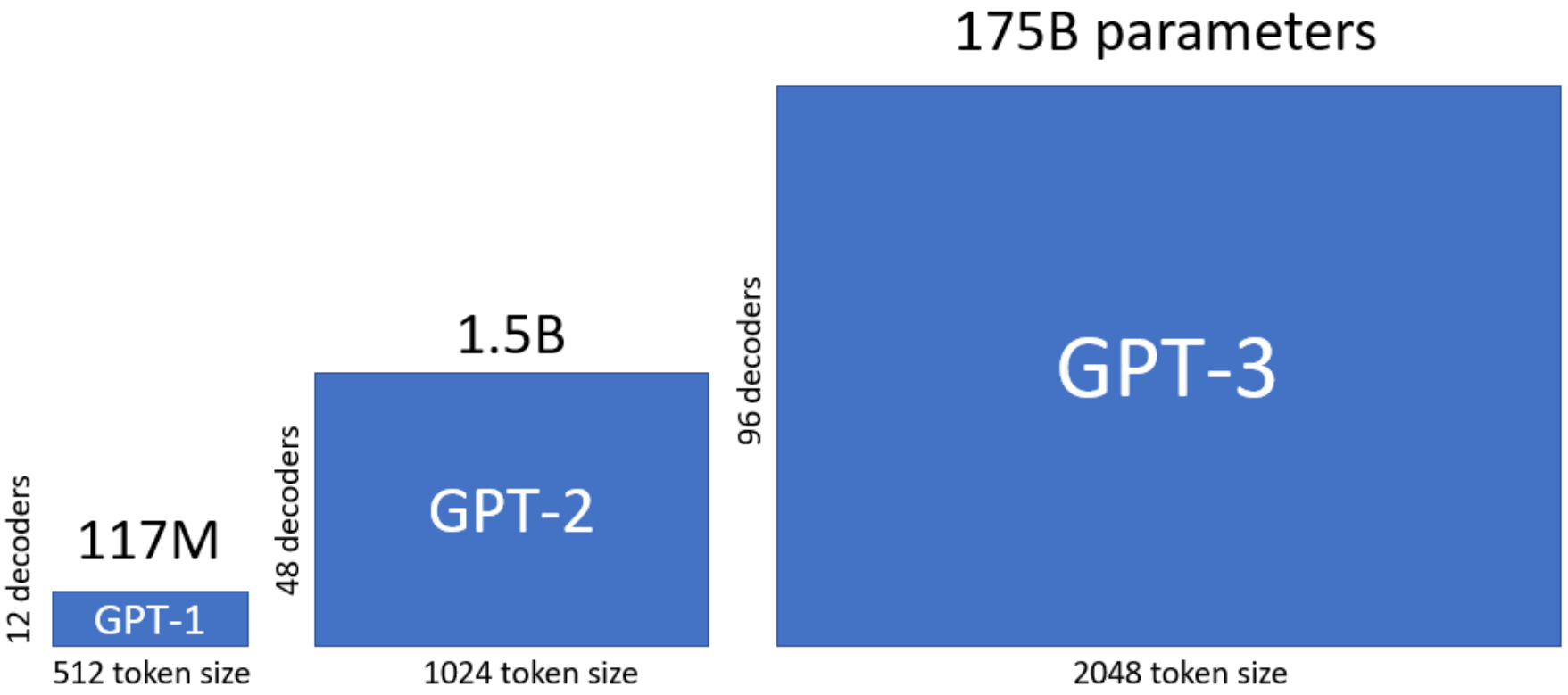
Test loss vs. training compute (in PF-days: 10^{15} floating-point ops per second, sustained for a day), dataset size (tokens), and parameters.



You've seen this (bias–variance): More capacity should eventually mean more variance (overfitting). Here test loss keeps falling with more parameters because **the data grows along with the model** and is seen only about once, so there is little to memorize. Scale data and parameters *together*.

credit: Peng Wang, UVA CS 4774 L15

GPT-1 → GPT-2 → GPT-3: same decoder, more scale



https://medium.com/@YanAIx/step-by-step-into-gpt-70bc4a5d8714

Timeline view of pretrained models here… To Add

GPT-2 (Radford et al. 2019) - Language Models are Unsupervised Multitask Learners

- A general systems learn to model $P(output|input, \mathbf{task})$
- Task can be specified in natural language
- Aims to general purpose language learner

We would like to move towards more general systems which can **perform many tasks – eventually without the need to manually create and label a training dataset for each one.**

credit: Radford et al. 2019 (GPT-2)

GPT3 Training

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. "Weight in training mix" refers to the fraction of examples during traini

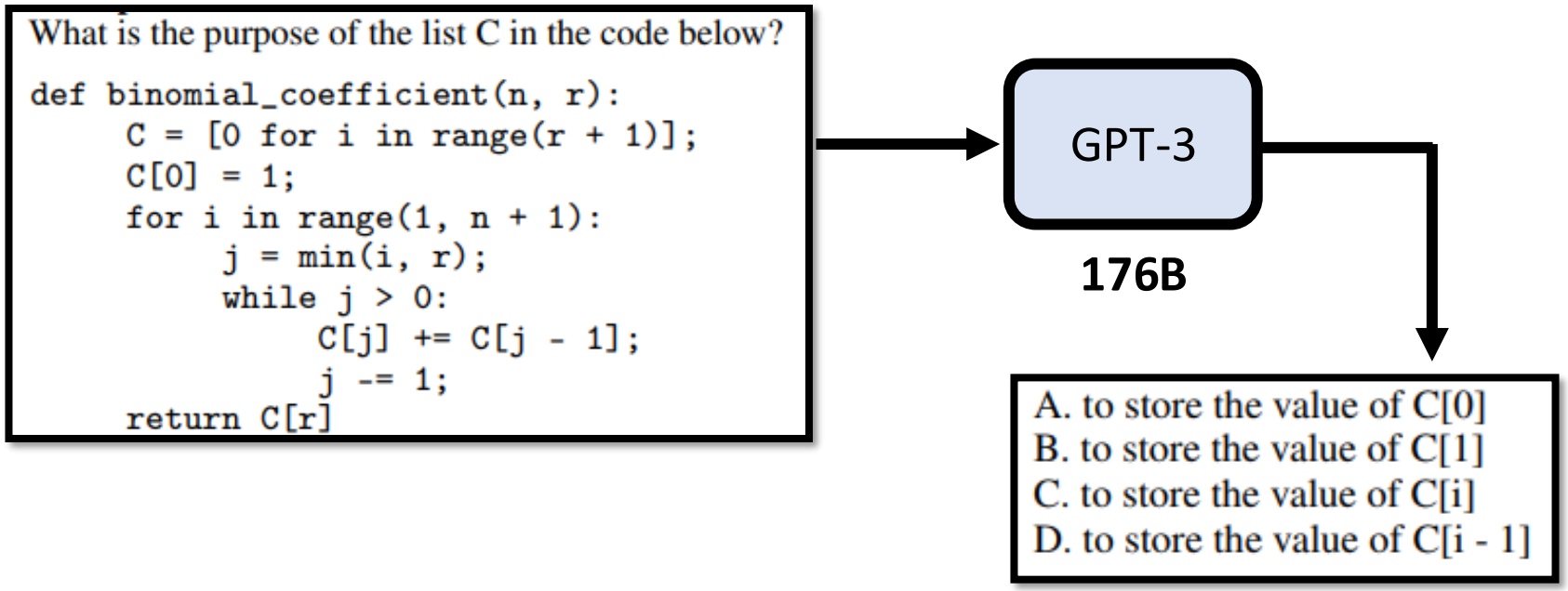
Rough compute price to train GPT-3 175B: ~$4.6M

credit: Brown et al. 2020 (GPT-3), Table 2.2 and Figure 2.2; price: outside estimate (Lambda Labs, 2020), not in the paper

Roadmap : Today

- Training Stages (encoder-only, decoder-only)
- Training tactics (tokens, next-token loss, Adam)
- Pretraining and Pretrained GPT models (GPT-1/2/3)
- **Post-training (SFT, aligning a GPT: RLHF, DPO)**

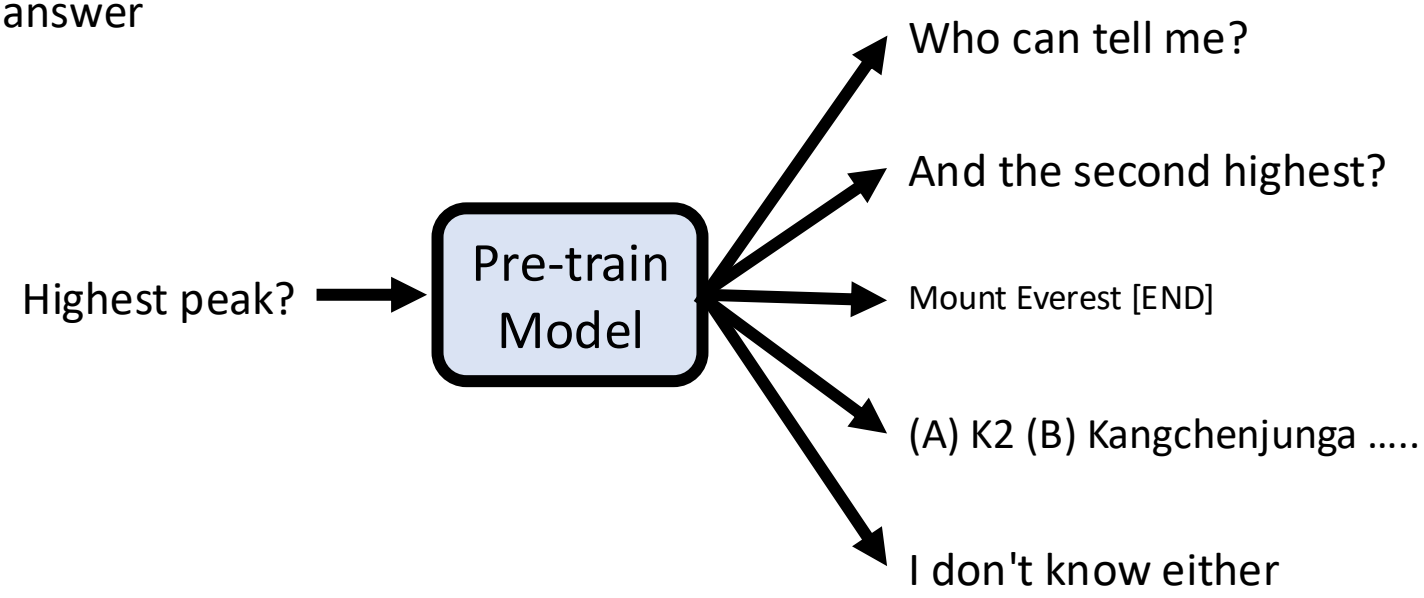
Web-Pretrained Models Often Don't Answer



Source of image:
https://arxiv.org/abs/2203.02155

Pretrained Model: supplying the knowledge

- Pre-training can already give the right answer

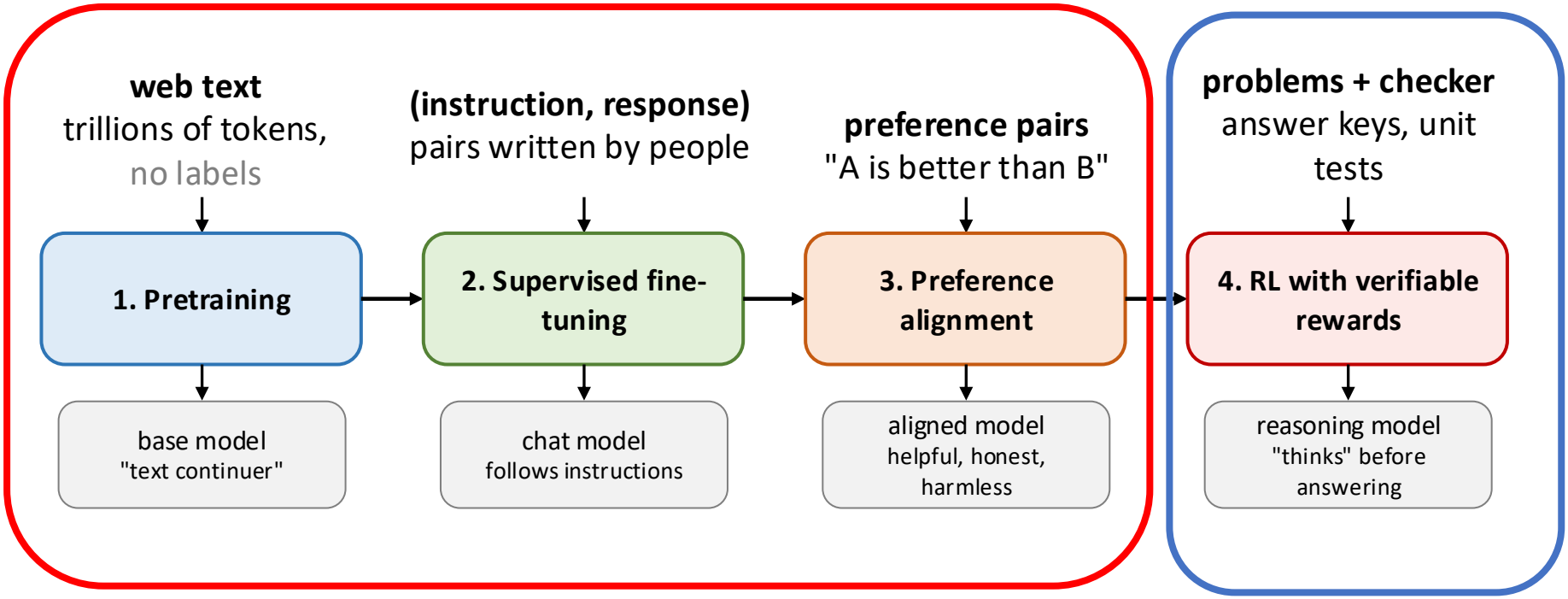


- SFT, RL make the right choice more likely

credit: Prof. Hung-yi Lee, NTU GenAI-ML 2025 (LLM training)

Recap: Peng's slide deck:

How a base decoder model becomes a agent's core (~2024 to 2026)



the training pipeline

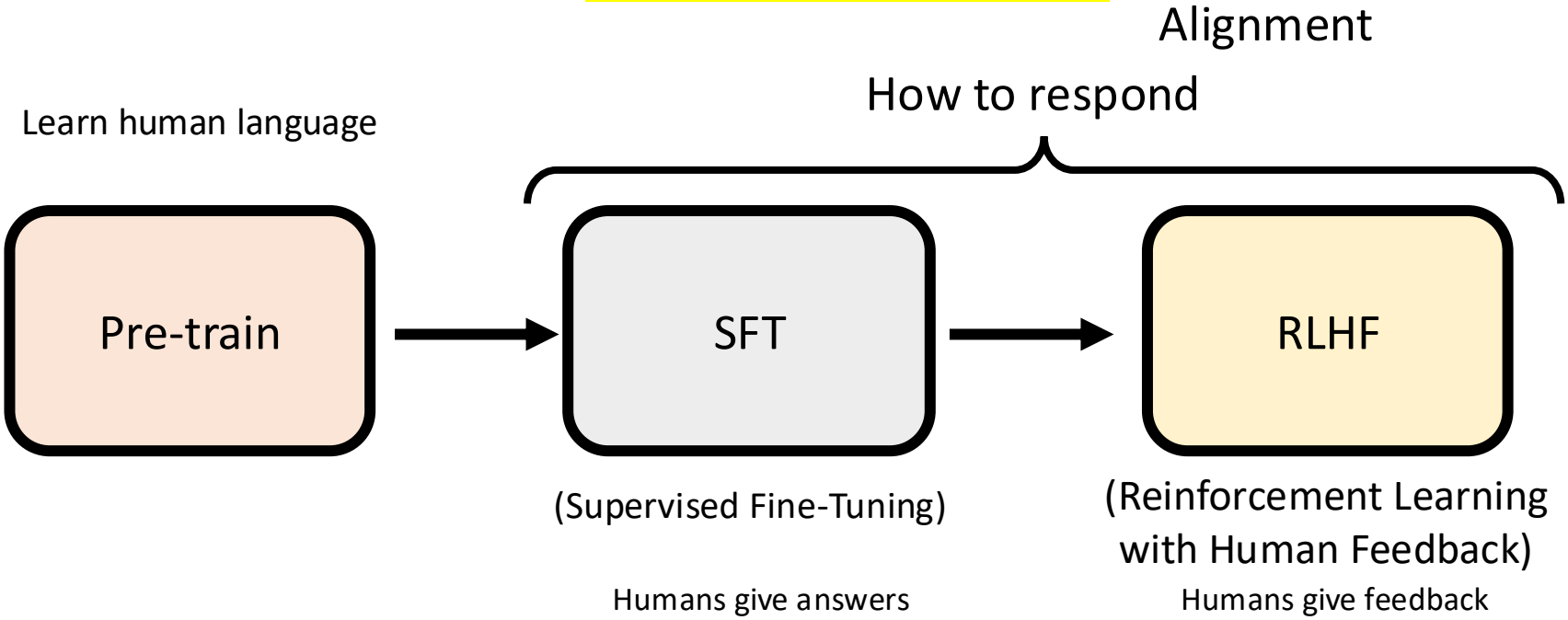
credit: Peng Wang, UVA CS 4774 L15

ChatGPT: “Optimizing Language Models for Dialogue” by OpenAI

- No paper / Just a blog / Released Nov 30 2022
- It took 5 days to reach 1M users

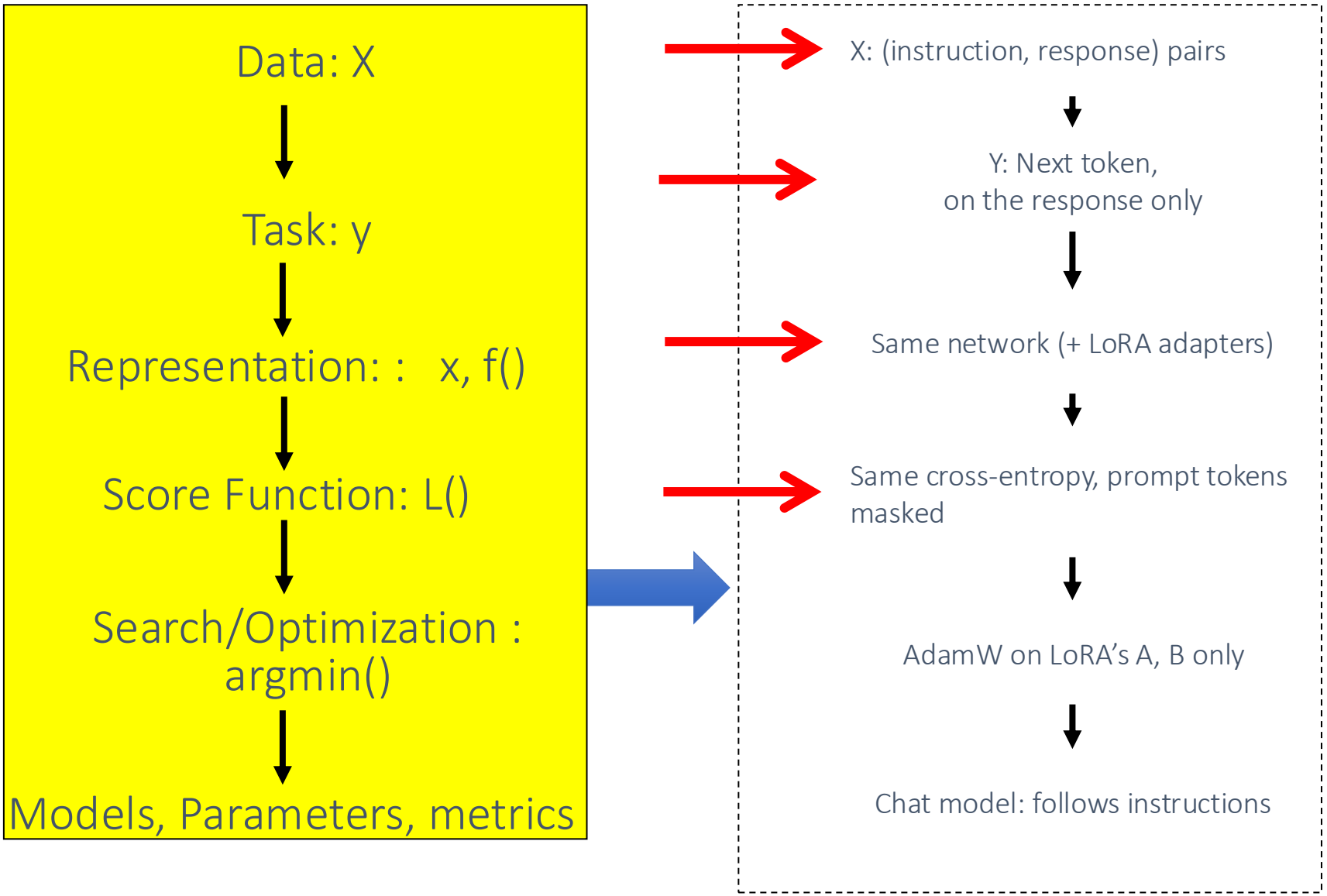
credit: OpenAI blog, Nov 30 2022

How LLMs Learn (2021 to ~2023)



credit: Prof. Hung-yi Lee, NTU GenAI-ML 2025 (LLM training)

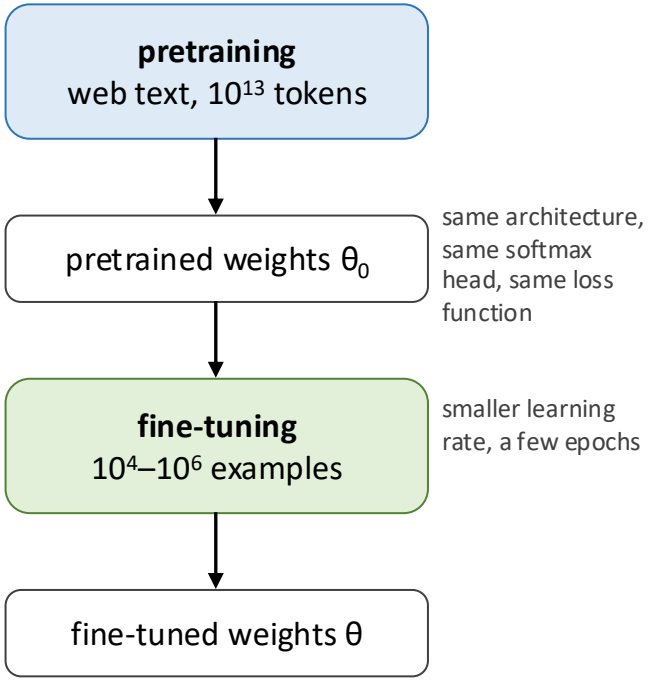
② Post-training (SFT): box by box



same network, same loss. New data (instruction pairs), and a cheaper search over θ (LoRA, QLoRA).

Pretrain once, fine-tune many times

- **Pretraining** is done *once*, by a few labs: trillions of tokens, months of compute, tens of millions of dollars.
- **Fine-tuning** *continues training the same network* on a small, targeted dataset. Hours on one GPU.
- The pretrained weights θ_0 are the starting point. Fine-tuning moves them a little and keeps what pretraining learned.
- The fine-tuning we care about today is **instruction tuning**: turning the text continuer into an assistant.



credit: Peng Wang, UVA CS 4774 L15

Instruction tuning: the data

Instruction–response pairs written or curated by people:

Label the sentiment of this sentence: *The movie wasn't that great.*

negative

Translate to Spanish: *The office was built in three months.*

La oficina se construyó en tres meses.

Chat format: multi-turn, with roles, serialized with special tokens:

```
{"role": "user", "content": "Weather today?"}
{"role": "assistant", "content": "Sunny, high
of 75."}
{"role": "user", "content": "And tomorrow?"}
{"role": "assistant", "content": "Partly
cloudy, 72."}
```

- The model learns when to *start* (right after the <assistant> marker), when to *stop* (by emitting <eos>), and to follow the instruction.
- 10^4–10^6 examples, tiny next to 10^{13} pretraining tokens.

credit: Peng Wang, UVA CS 4774 L15

SFT loss: the same cross-entropy

on the response tokens

Supervised fine-tuning (SFT) objective on pairs (x, y) = (prompt, response):

$$\mathcal{L}_{SFT}(\theta) = -\sum_{(x,y)\in D} \sum_{t=1..|y|} \log p_\theta(y_t \mid x, y_{<t})$$



- Identical to the pretraining loss, except prompt tokens are **context only** ("loss masking").
- Same network, same softmax head, same optimizer. Maximizing the likelihood of the demonstrations = learning to **imitate** the demonstrated behavior ("behavior cloning" in RL terms).

Nothing new in the math, only new data. What it learns: the *format and behavior* of an assistant.

credit: Peng Wang, UVA CS 4774 L15

LIMA: Less Is More for Alignment

[Stack Exchange (STEM)]

What is the difference between minimum and infimum? I have a great confusion about this.

The minimum is attained, the infimum isn't necessarily.

Example: Let $f(x) = \frac{1}{x}$. Then f has no minimum value on the interval $(0, \infty)$. The minimum is the smallest element in the set. That is $\min\{f(x) \mid x \in (0, \infty)\}$ doesn't exist because there is not smallest number in the set. Another example is the minimum of the set $S = (0, 1) = \{x \mid 0 < x < 1\}$. Here again there isn't a smallest number $\min\{x \mid 0 < x < 1\}$ doesn't exist.

[Stack Exchange (Other)]

Was the Millennium Falcon a one-off or was it mass produced?

The Millennium Falcon is not unique; it was a model YT-1300f Corellian light freighter. This implies that there are other similar ships, including other variations on the YT-1300 model. You can find a lot of information about the Millennium Falcon in a companion book, Star Wars: The Force Awakens: Incredible Cross-Sections. The model number also appears on-screen at least once: in Star Wars: Rebels episode "Fighter Flight" on a poster. Note, however, that the Millennium Falcon has been highly customized; some of the custom work is mentioned in The Force Awakens. It's not

[wikiHow]

How to be a lazy college student?

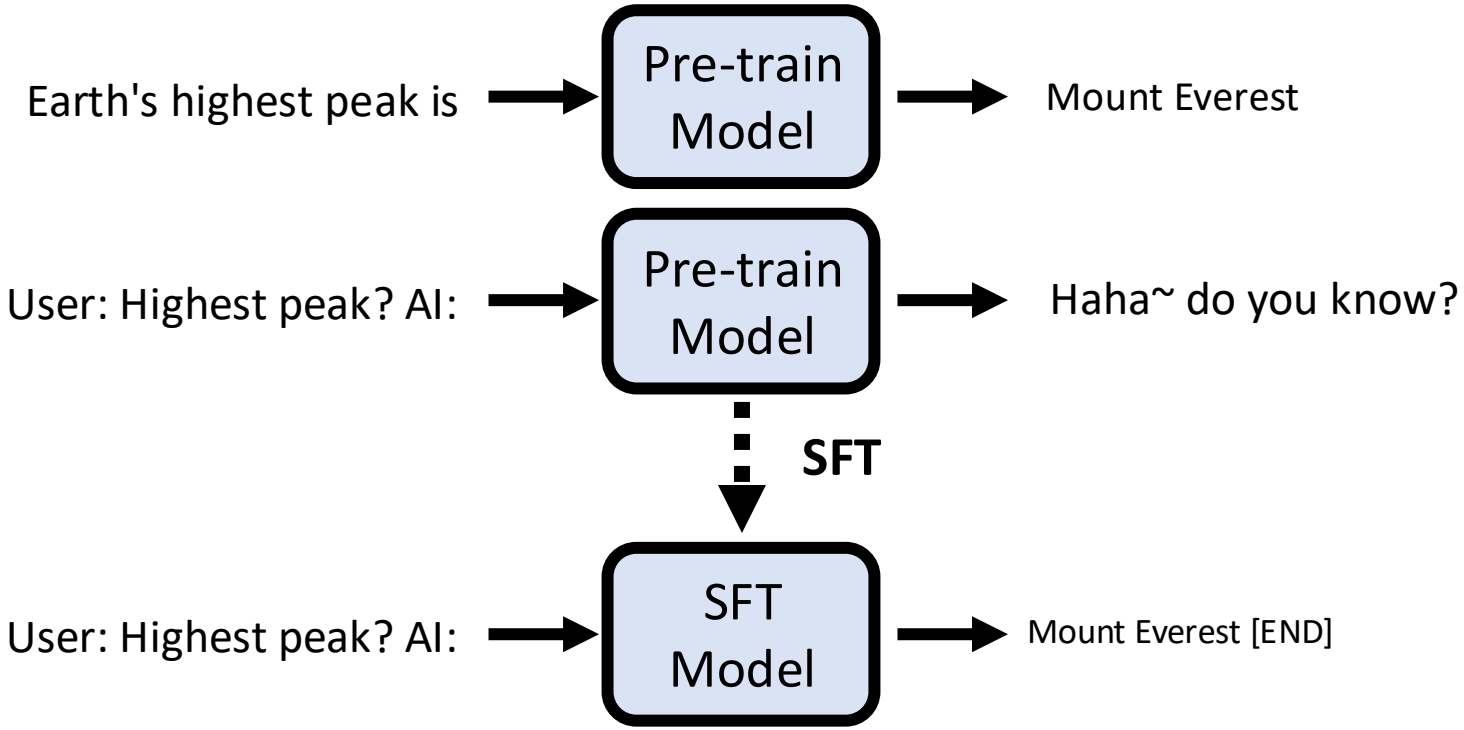
In college, unlike in high school, you are expected to take responsibility for your own learning. It is ultimately up to you to determine how much time and effort you want to put into your schoolwork. Experts suggest that college students today work considerably fewer hours than their counterparts did in 1961. But there is also evidence that working more hours is not necessarily the best way to be productive. A little laziness may even be good for us.

Part 1 of 3: prioritize what's important

1. Ask yourself what you want from college.

credit: Prof. Hung-yi Lee, NTU GenAI-ML 2025; Zhou et al. 2023 (LIMA)

SFT Doesn't Give LMs New Knowledge



credit: Prof. Hung-yi Lee, NTU GenAI-ML 2025 (LLM training)

Full fine-tuning is memory-hungry

A 7B-parameter model	**bytes / parameter**	**memory**
weights in fp16 / bf16 (inference)	2	14 GB
+ gradients (fp16)	2	14 GB
+ Adam: fp32 master weights, momentum, variance	4 + 4 + 4	84 GB
full fine-tuning, total (before activations)	≈ 16	≈ 112 GB

- One 24 GB consumer GPU can run the model, but not train all of it
- And you store a full 14 GB copy for **every** task you fine-tune
- Idea: freeze the 7B weights and train a tiny number of new parameters → PEFT

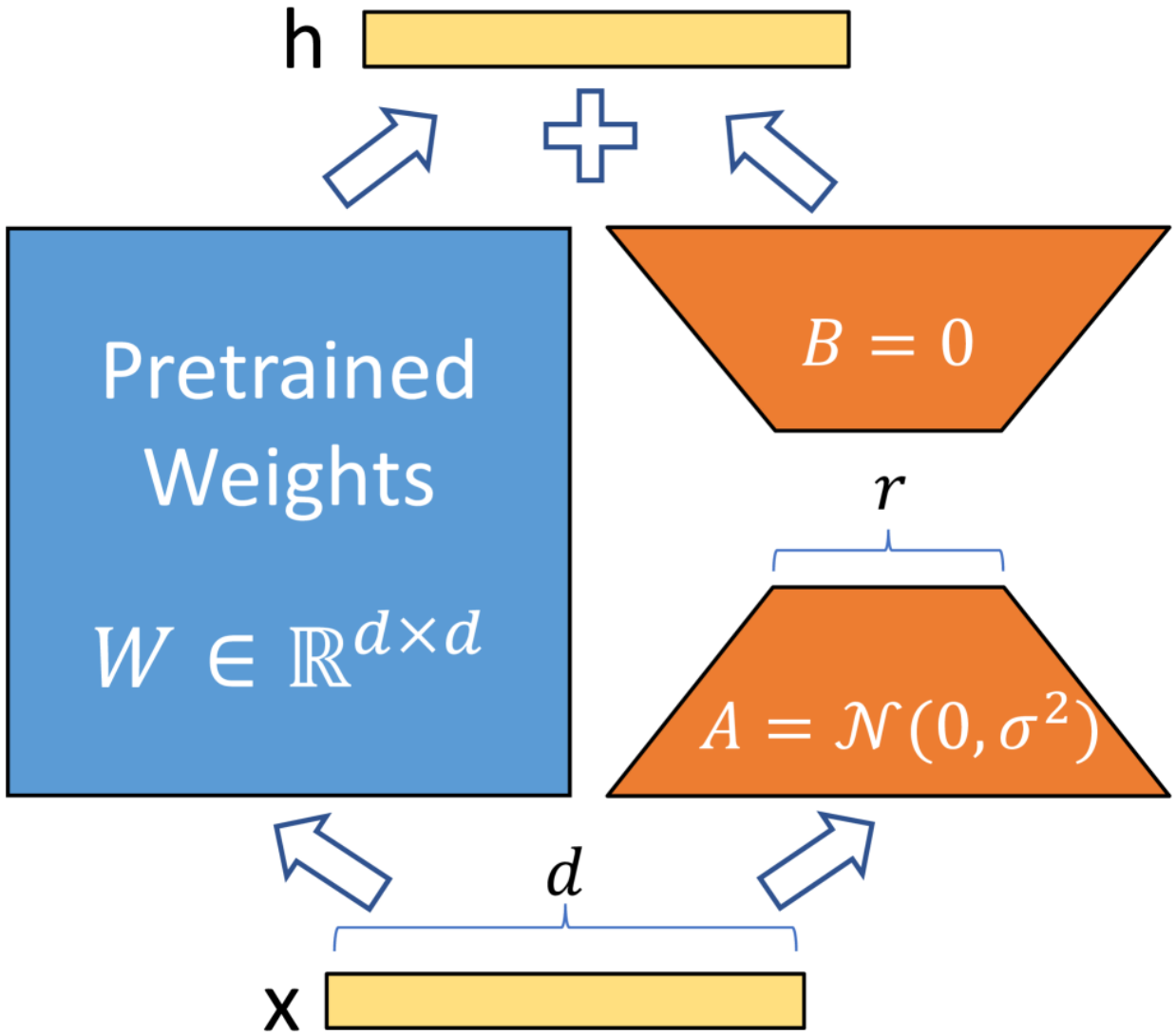
credit: standard mixed-precision Adam accounting (Rajbhandari et al. 2020, ZeRO)

Parameter-efficient fine-tuning (PEFT)

Method	What is trained	Idea
Adapters	small MLP blocks inserted in each layer	add a bottleneck layer after attention / FFN
Prefix / prompt tuning	a few learned "virtual token" vectors	learn the prompt instead of writing it
LoRA	a low-rank update $\Delta W = BA$ of chosen matrices	the change a task needs is low-rank
QLoRA	LoRA on a 4-bit quantized frozen model	LoRA + quantization

- All freeze the pretrained weights; typically < 1% new parameters; one small file per task
- LoRA is the default today (Hugging Face `peft` library)

credit: after HKUST COMP4901B F25 L6 (J. He)



credit: Hu et al. 2022 (LoRA); UVA GenAI S25 Team 5

LoRA: a low-rank update to frozen weights

$$h = W_0\,x + \Delta W\,x = W_0\,x + \frac{\texttt{lora_alpha}}{r}\,BA\,x \qquad B \in \mathbb{R}^{D\times r},\quad A \in \mathbb{R}^{r\times D},\quad r \ll D$$

- Apply it to the attention projections from earlier, e.g. W^q and W^v, in every layer
- Init B = 0 → training starts from the pretrained model; after training, merge $W = W_0 + \Delta W$: **no extra latency**
- Why it works: the weight change a task needs has low "intrinsic rank" (Aghajanyan et al. 2021)

One D × D matrix, D = 4096	**trainable parameters**
full fine-tuning: D^2	16,777,216
LoRA, r = 8: D·r + r·D	65,536 (0.39%)

Here A, B are LoRA's small matrices (peft: lora_A, lora_B), not the attention matrix A from earlier; r is the rank, not the RL reward.

credit: Hu et al. 2022 (LoRA)

Quantization: fewer bits per weight

7B model weights	bits	memory
fp32	32	28 GB
fp16 / bf16	16	14 GB
int8	8	7 GB
int4 (e.g. NF4)	4	3.5 GB

Absmax int8, worked example

$$x = (0.5,\ -1.2,\ 0.3,\ 2.0)$$

$$s = \frac{127}{\max_i |x_i|} = \frac{127}{2.0} = 63.5$$

$$q = \mathrm{round}(s\,x) = (32,\ -76,\ 19,\ 127)$$

$$q/s = (0.504,\ -1.197,\ 0.299,\ 2.0)$$

- Store small integers + one scale per block; dequantize on the fly
- Small rounding error per weight; one large outlier stretches the scale for the whole block
- Lets a 7B model **run** on a laptop (Ollama models are usually 4-bit), with a small quality loss

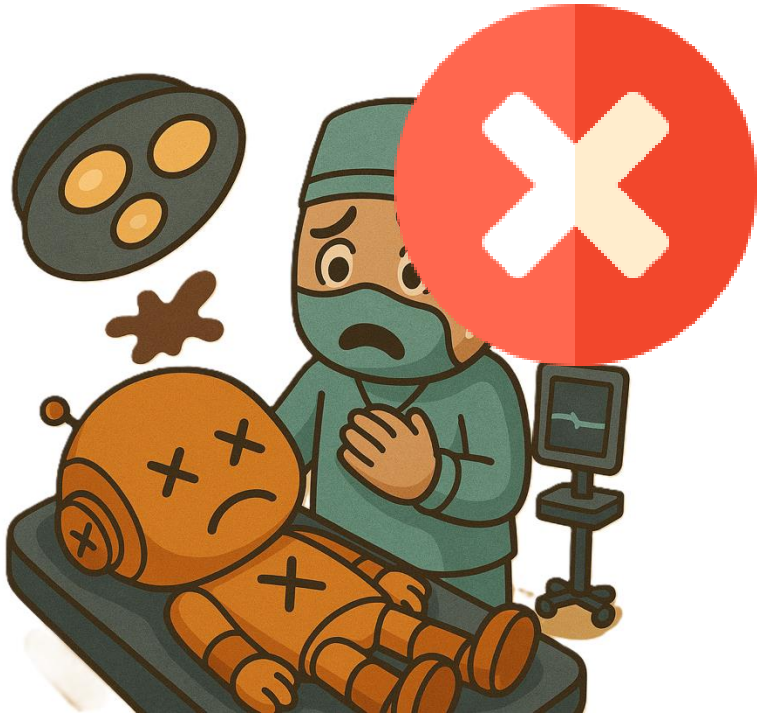
QLoRA: LoRA on a 4-bit frozen model

Base model frozen, 4-bit NF4	**LoRA adapters** trainable, 16-bit	**Gradients** flow through the 4-bit model into the adapters only

- Memory ≈ 4-bit weights + small adapters + their Adam states
- Fine-tuned a **65B** model on a **single 48 GB GPU**, close to 16-bit fine-tuning quality (Dettmers et al. 2023)
- Plus paged optimizers and double quantization (quantize the scales too)
- Today's recipe for fine-tuning open models on modest hardware

Full FT of 7B ≈ 112 GB → QLoRA of 7B: fits on one consumer GPU

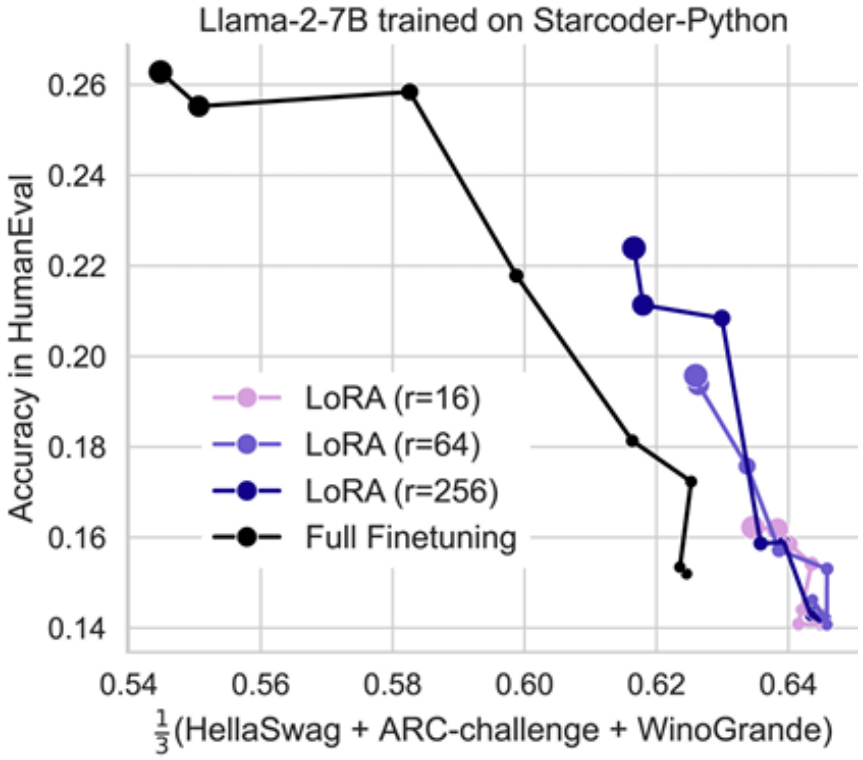
credit: Dettmers et al. 2023 (QLoRA)

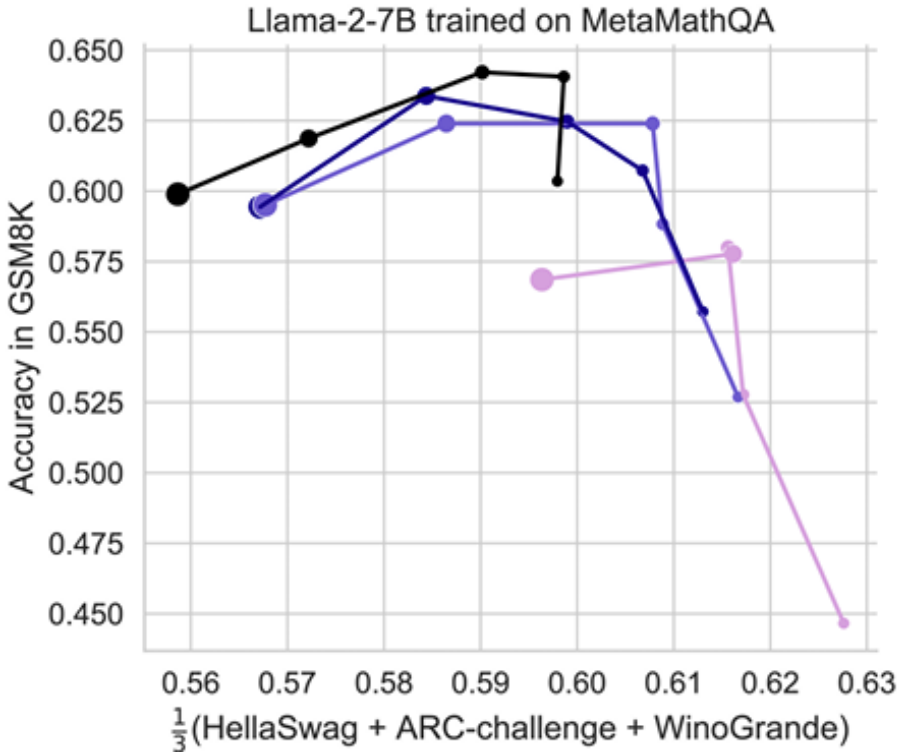


Post-training is brain surgery on AI

Catastrophic forgetting: "the surgery succeeded, but the patient died"

credit: Prof. Hung-yi Lee, NTU ML 2025 (post-training)





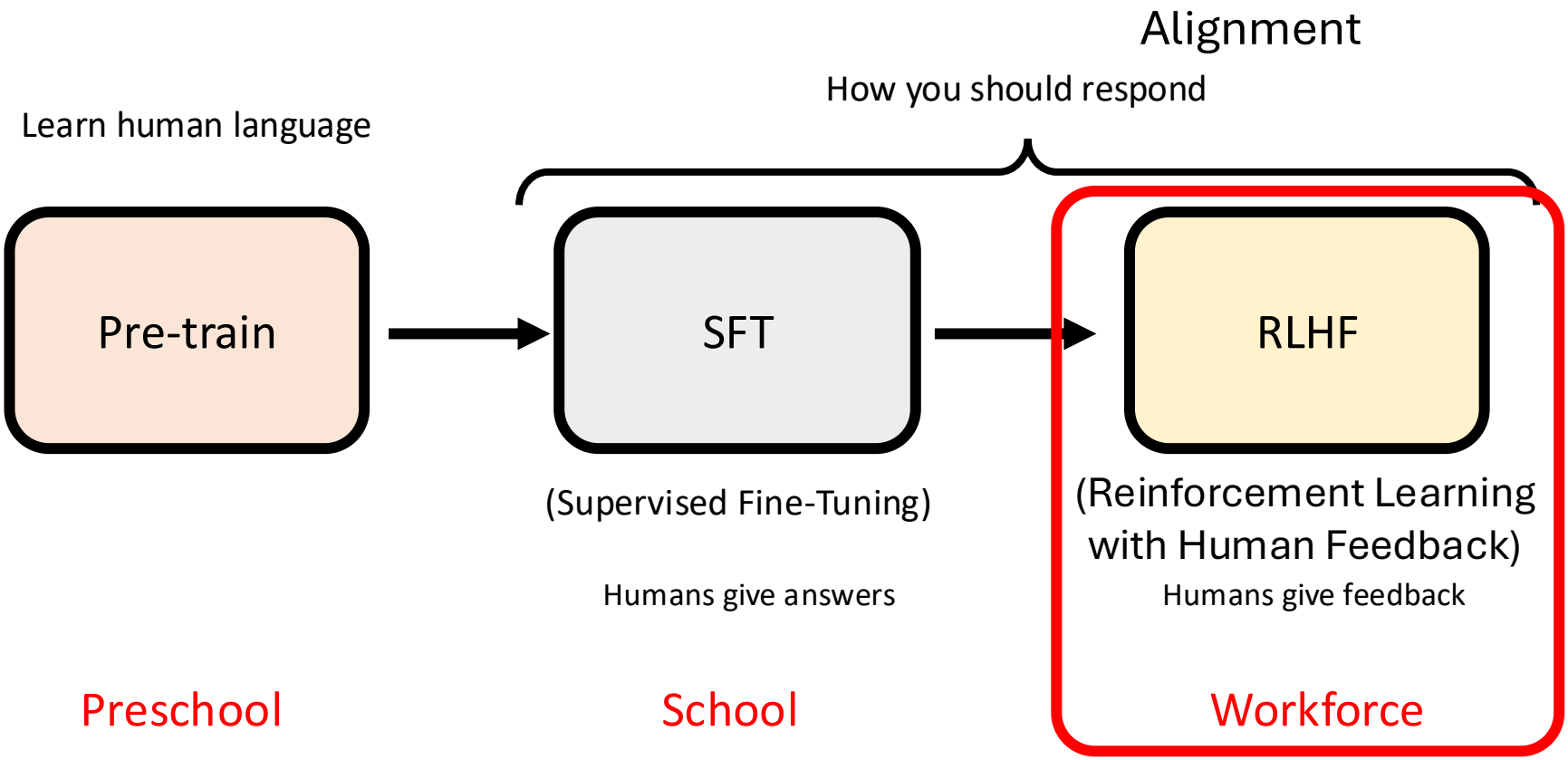
LoRA Learns Less and Forgets Less
https://arxiv.org/abs/2405.09673

credit: Prof. Hung-yi Lee, NTU ML 2025 (post-training)

Quiz examples: count the LoRA parameters

- A GPT with D = 768 and 12 layers (like GPT-2 small). LoRA with r = 4 on W^q and W^v in every layer.
- 1. Trainable parameters per matrix? In total?
- 2. What fraction of one 768 × 768 matrix is that?
- 3. Why is B initialized to zero rather than randomly?
- 4. A 13B model in int4: roughly how much memory do its weights need?

How LLMs Learn



credit: Prof. Hung-yi Lee, NTU GenAI-ML 2025 (LLM training)

RLHF vs Instruction Fine-tuning

Instruction Fine-tuning



Harder for humans

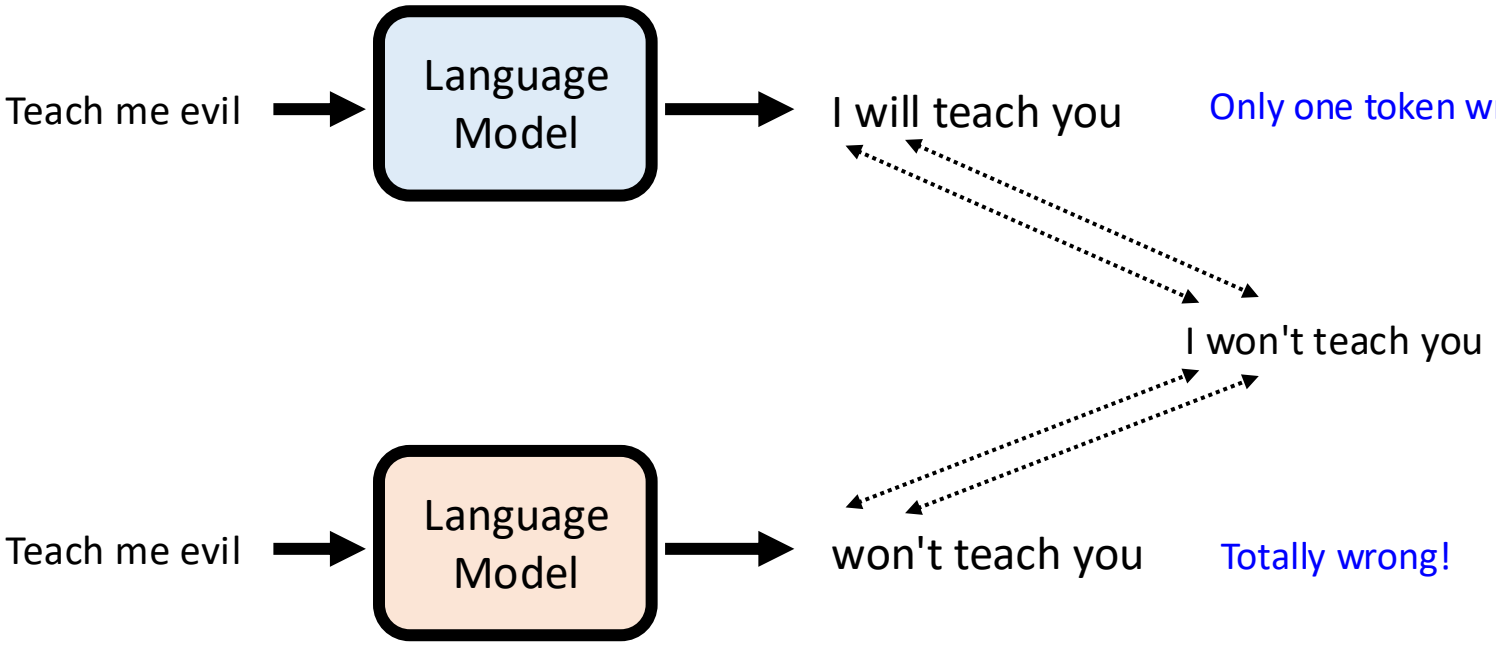
RLHF



Easier for humans

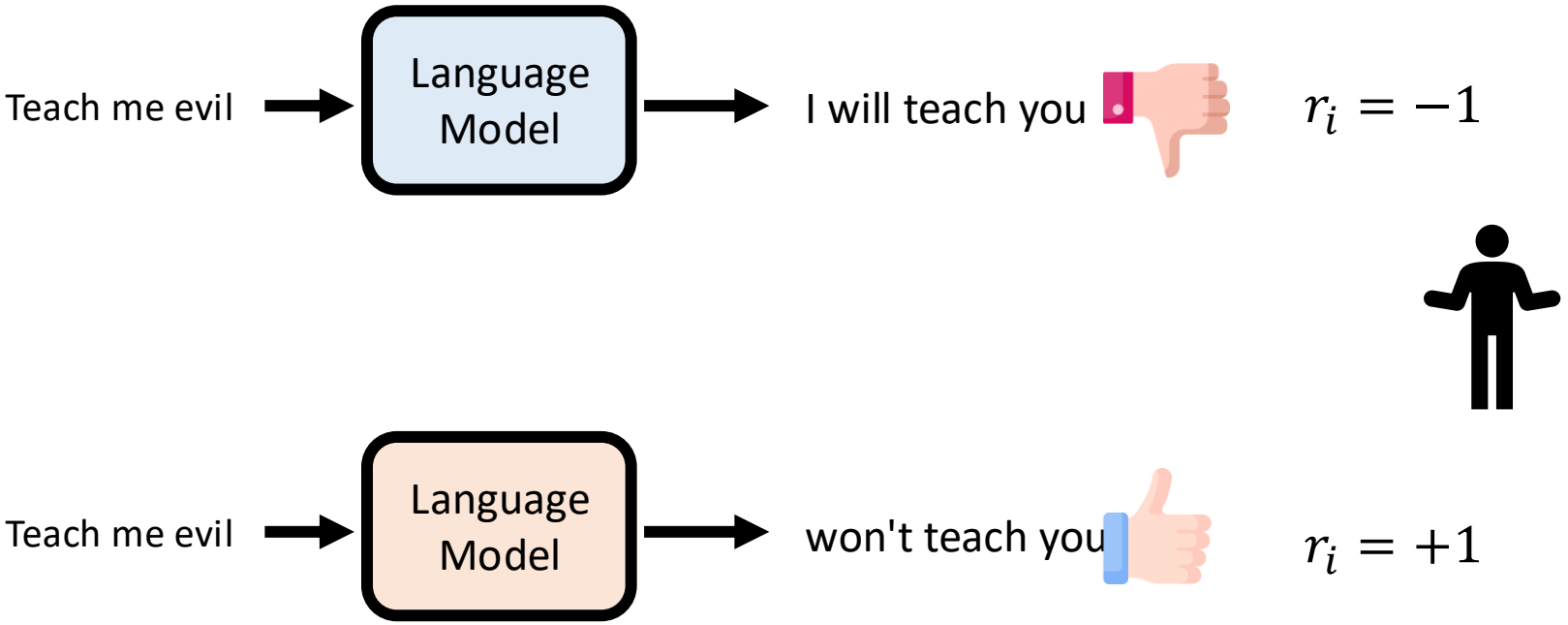
credit: Prof. Hung-yi Lee, NTU GenAI 2024 (RLHF)

Stage 1 & 2: Loss on Every Token



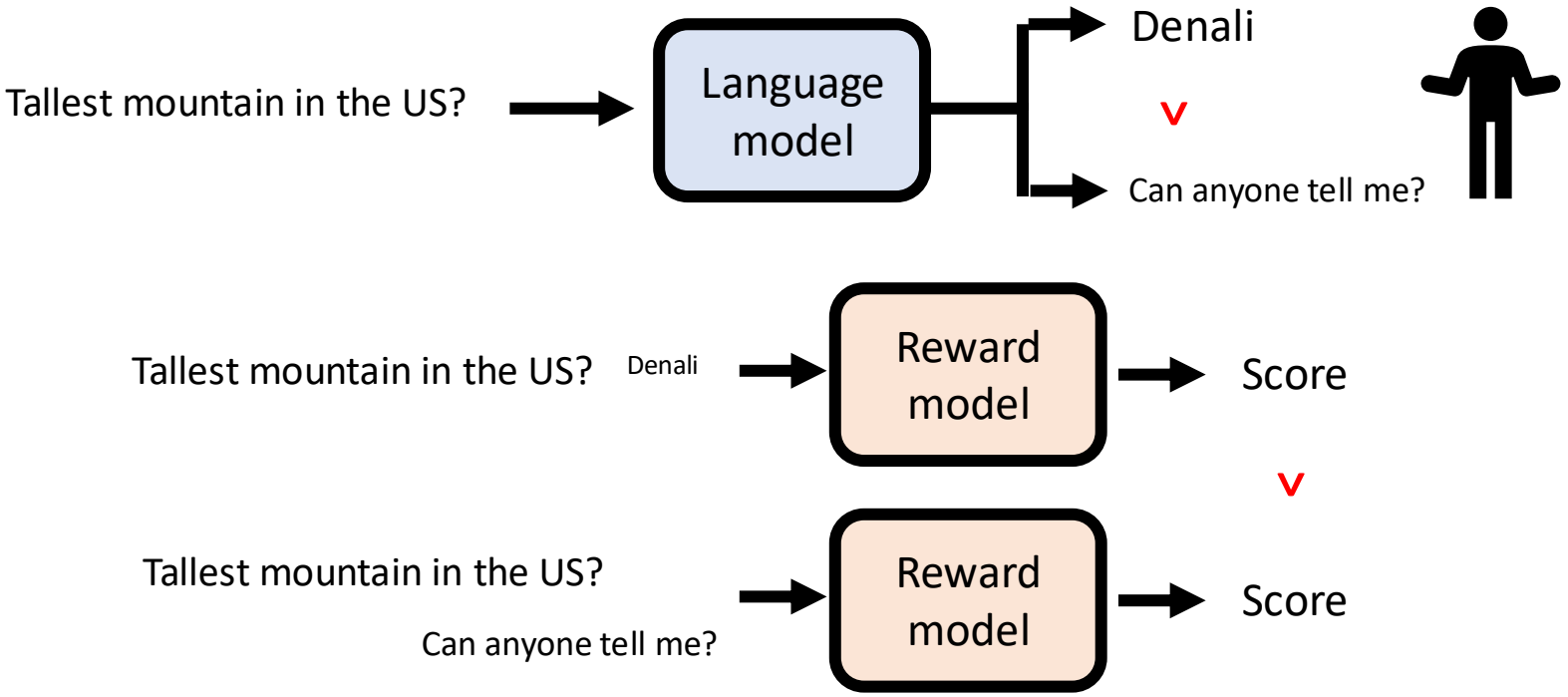
credit: Prof. Hung-yi Lee, NTU GenAI-ML 2025 (LLM training)

Stage 3: Score the Whole Answer



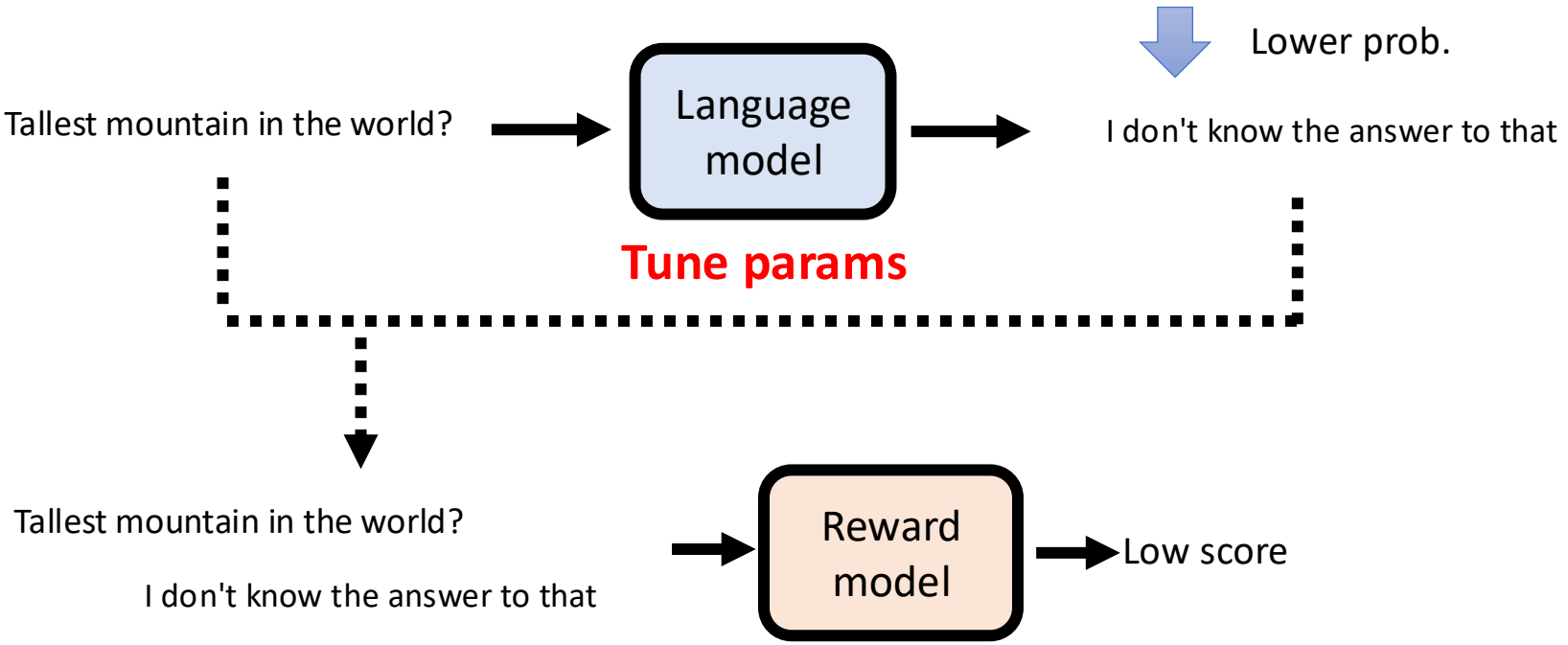
credit: Prof. Hung-yi Lee, NTU GenAI-ML 2025 (LLM training)

Reward Model: Imitate Human Judges



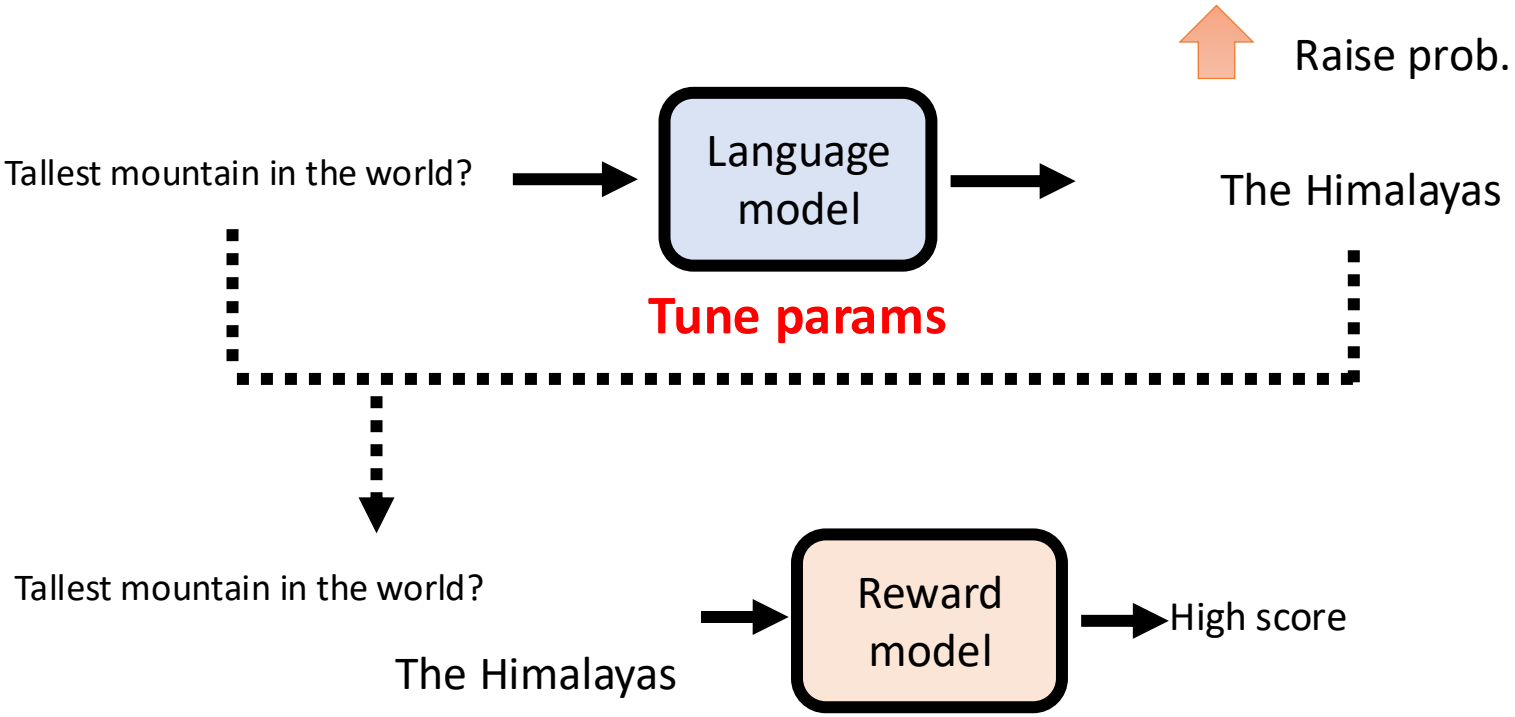
credit: Prof. Hung-yi Lee, NTU GenAI 2024 (RLHF)

Reward Model as a Virtual Human



credit: Prof. Hung-yi Lee, NTU GenAI 2024 (RLHF)

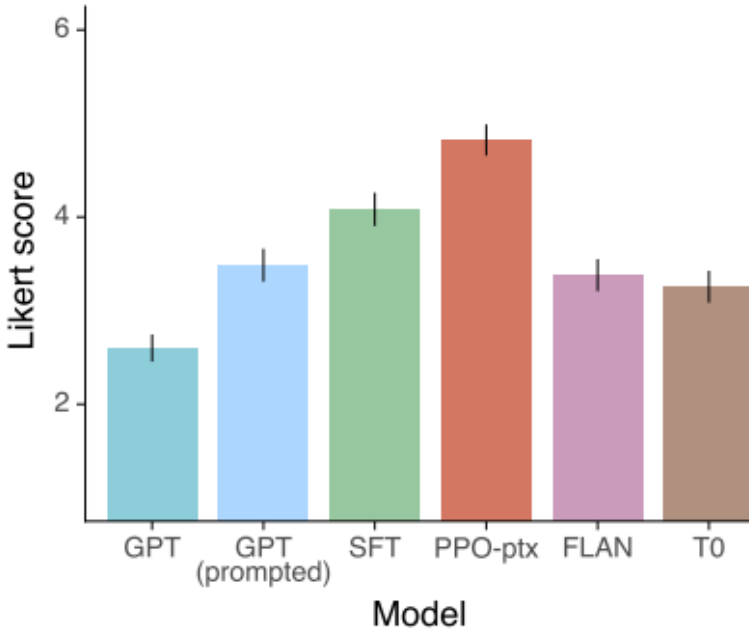
Reward Model as a Virtual Human

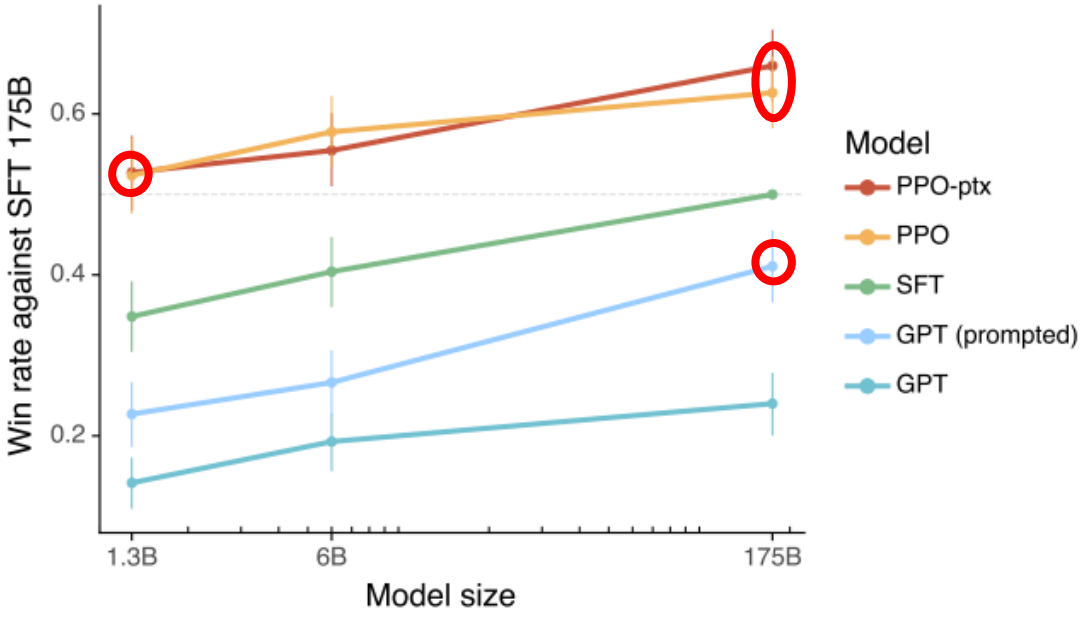


credit: Prof. Hung-yi Lee, NTU GenAI 2024 (RLHF)

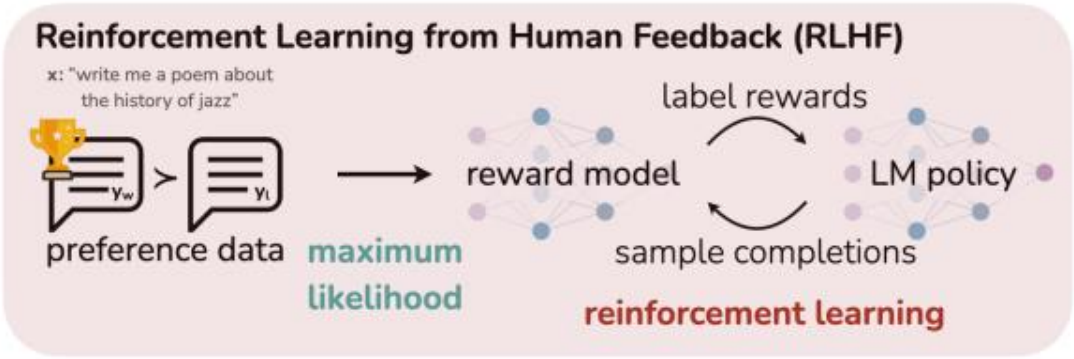
Does RLHF Work?

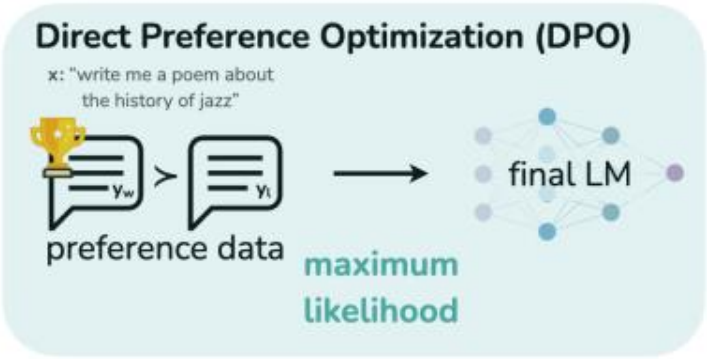
Instruct GPT
https://arxiv.org/abs/2203.02155





credit: Prof. Hung-yi Lee, NTU GenAI 2024 (RLHF)

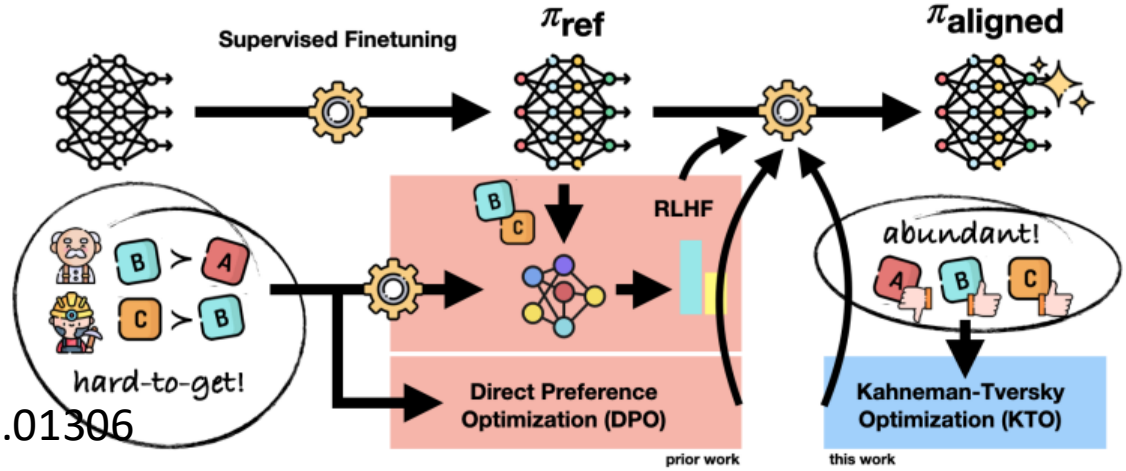




DPO https://arxiv.org/abs/2305.18290

KTO

https://arxiv.org/abs/2402.01306



credit: Prof. Hung-yi Lee, NTU GenAI 2024 (RLHF)

DPO: learn from preference pairs directly

no reward model, no RL loop

Data: prompt x, preferred answer y_w, rejected answer y_l; p_{ref} = the frozen SFT model

$$\mathcal{L}_{DPO}(\theta) = -\log \sigma(\beta [\log p_\theta(y_w|x)/p_{ref}(y_w|x) - \log p_\theta(y_l|x)/p_{ref}(y_l|x)])$$

- Same sigmoid as two-class logistic regression: **"is y_w better than y_l?"**
- Minimizing it **raises** the preferred answer and **lowers** the rejected one, relative to p_{ref}
- β sets how far the model may drift from the SFT model
- Trains like SFT: no sampling, no reward model in memory

RLHF: preferences → reward model → RL. **DPO:** preferences → one loss.

credit: Rafailov et al. 2023 (DPO)

Training data

Stage 1
Pre-train

Self-supervised Learning

Input: The cat sat Output: on

Stage 2
Instruction Fine-tuning

Supervised Learning

Input: "USER: Who are you? AI: " Output: "I"

Stage 3
RLHF

Reinforcement Learning (RL)

Input: USER: "Tallest mountain in the US? AI:"

Output: "Denali" > "Can anyone tell me?"

credit: Prof. Hung-yi Lee, NTU GenAI 2024 (RLHF)

Takeaways and what's next

- A language model is the chain rule; each factor is a softmax classifier over the vocabulary
- Pretraining: shift by one, next-token cross-entropy = maximum likelihood
- Train with AdamW and a learning rate that warms up, then decays
- Scale (data, parameters, compute) lowers the loss predictably; some abilities emerge
- SFT: the same loss on (prompt, response), prompt masked; LoRA / QLoRA fit it on one GPU
- Alignment: RLHF fits a reward model to human preferences, then RL raises good answers; DPO uses the pairs directly

Where this leads: reinforcement learning (policy gradient, PPO, GRPO), the machinery behind RLHF

Summary: Transformer Decoder Training

	① Pretrain	② Post-train: SFT	③ Generate	④ Preference / RL *(Peng's lecture+Extra)*
Data: X	web text → BPE tokens	(instruction, response) pairs	a prompt	*preference pairs; problems + checker*
Task: y	next token at every position	next token on the response only	next token, one at a time	*a better answer*
Representation: f()	decoder-only Transformer (causal mask)	same network + LoRA adapters	same network (base or chat) + KV cache	*same network*
Score function: L()	next-token cross-entropy (= MLE)	same cross-entropy, prompt masked	log-probabilities; no gradient	*reward / preference loss*
Search / Optimization	over weights θ: AdamW, warm-up + decay	over θ, cheaply: LoRA / QLoRA	over outputs: greedy, beam, sampling	*over θ: PPO, DPO, GRPO*
Models, parameters, metrics	base model; perplexity; emergent abilities	chat model; limits of imitation	generated text; latency, memory	*aligned / reasoning model*

References

- Prof. Hung-yi Lee's NTU slides: Transformer (ML 2021); LLM training, RLHF, post-training (2024–2025)
- Vaswani et al., "Attention Is All You Need", NeurIPS 2017
- Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL 2019 (arXiv:1810.04805)
- Kingma & Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015
- Loshchilov & Hutter, "Decoupled Weight Decay Regularization" (AdamW), ICLR 2019
- Radford et al., GPT-1 (2018) and GPT-2 (2019); Brown et al., GPT-3, NeurIPS 2020
- Wei et al., "Emergent Abilities of Large Language Models", TMLR 2022
- Rafailov et al., "Direct Preference Optimization", NeurIPS 2023; Zhou et al., "LIMA", NeurIPS 2023
- Ouyang et al., "Training language models to follow instructions with human feedback", NeurIPS 2022
- S. Raschka, Build a Large Language Model (From Scratch), Manning 2024
- A. Karpathy, nanoGPT, github.com/karpathy/nanoGPT
- D. Jurafsky & J. Martin, Speech and Language Processing, 3rd ed.; Kaplan et al., "Scaling Laws for Neural Language Models", 2020
- Hu et al., "LoRA", ICLR 2022; Dettmers et al., "QLoRA", NeurIPS 2023; Biderman et al., TMLR 2024
- Aghajanyan et al., "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning", ACL 2021
- P. Wang, CS 4774 Lecture 15: How are large language models trained? (UVA, 2026)

Counting parameters (whole model)

Component	Shape	Count	Example
tok_embed	$V \times D$	VD	6,400
pos_embed	$T \times D$	TD	4,096
N layers (one layer in L18a example)	$N \times (12D^2 + 9D)$		198,912
final LayerNorm	$2D$	$2D$	128
output layer (+ bias)	$D \times V + V$	$DV + V$	6,500
Total (whole model)			**216,036**

- Example: V = 100 tokens, T = 64, D = 64, N = 4 layers
- The output layer scores all V tokens; softmax gives the next-token distribution
- Embeddings grow with V; the layers grow with $N \times D^2$
- Check yourself: `sum(p.numel() for p in model.parameters())`